

Neural Networks

EEE 443/543

Lecture 10 -
Autoencoders

Principal component analysis

Principal component analysis (PCA) is a popular technique for analyzing large datasets containing a high number of dimensions/features per observation, increasing the interpretability of data while preserving the maximum amount of information, and enabling the visualization of multidimensional data. Formally, PCA is a statistical technique for reducing the dimensionality of a dataset. This is accomplished by linearly transforming the data into a new [coordinate system](#) where (most of) the variation in the data can be described with fewer dimensions than the initial data.

https://en.wikipedia.org/wiki/Principal_component_analysis

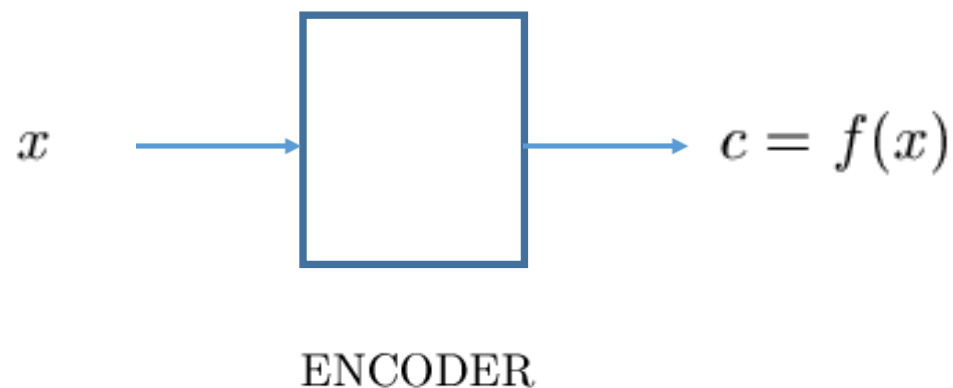
Suppose we have a collection of m points $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ in $\mathbb{R}^n$. Suppose we would like to apply lossy compression to these points. Lossy compression means storing the points in a way that requires less memory but may lose some precision. We would like to lose as little precision as possible.

Suppose we have a collection of m points $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ in $\mathbb{R}^n$. Suppose we would like to apply lossy compression to these points. Lossy compression means storing the points in a way that requires less memory but may lose some precision. We would like to lose as little precision as possible.

$$\mathbf{x}^{(i)} \in \mathbb{R}^n \quad \longrightarrow \quad \mathbf{c}^{(i)} \in \mathbb{R}^l \quad \longrightarrow \quad \hat{\mathbf{x}}^i \approx \mathbf{x}^i \in \mathbb{R}^n$$

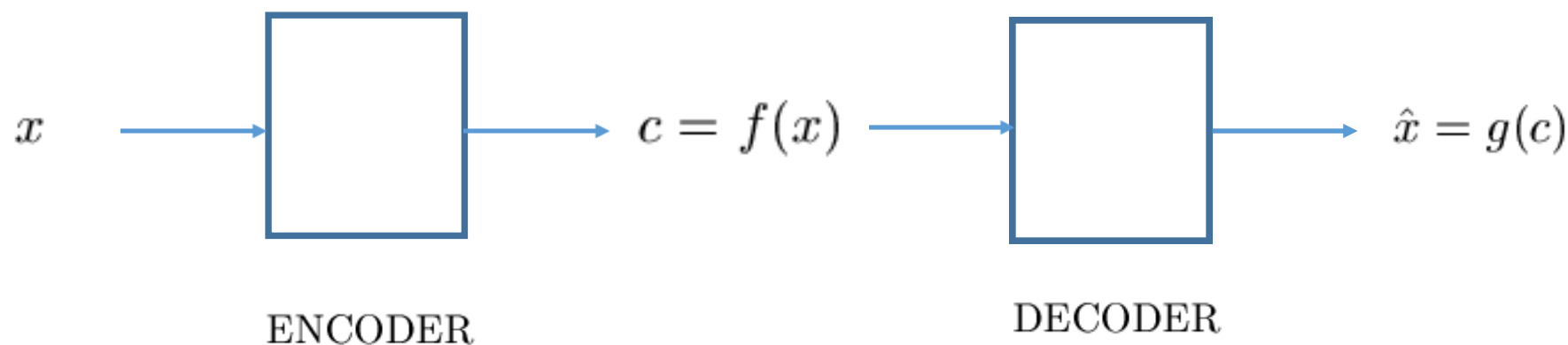
Suppose we have a collection of m points $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ in $\mathbb{R}^n$. Suppose we would like to apply lossy compression to these points. Lossy compression means storing the points in a way that requires less memory but may lose some precision. We would like to lose as little precision as possible.

$$\mathbf{x}^{(i)} \in \mathbb{R}^n \quad \longrightarrow \quad \mathbf{c}^{(i)} \in \mathbb{R}^l \quad \longrightarrow \quad \hat{\mathbf{x}}^i \approx \mathbf{x}^i \in \mathbb{R}^n$$



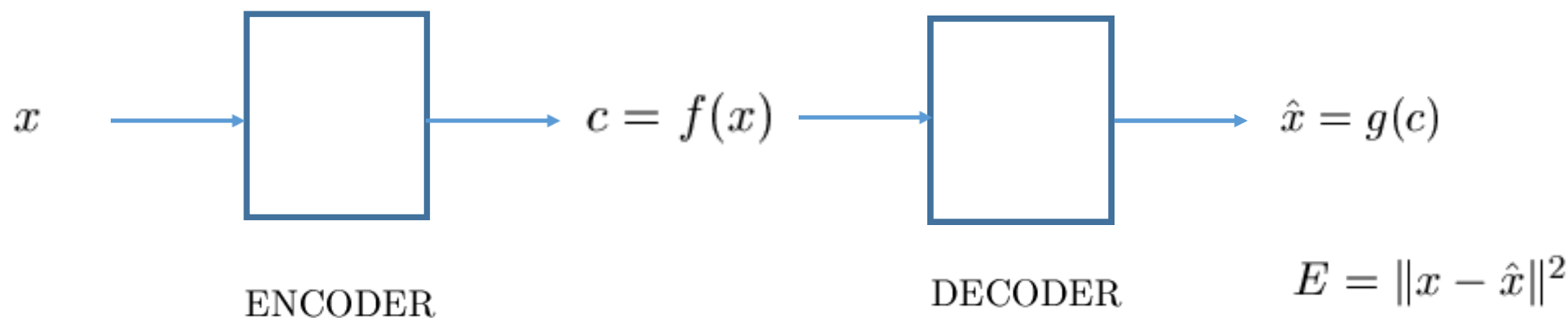
Suppose we have a collection of m points $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ in $\mathbb{R}^n$. Suppose we would like to apply lossy compression to these points. Lossy compression means storing the points in a way that requires less memory but may lose some precision. We would like to lose as little precision as possible.

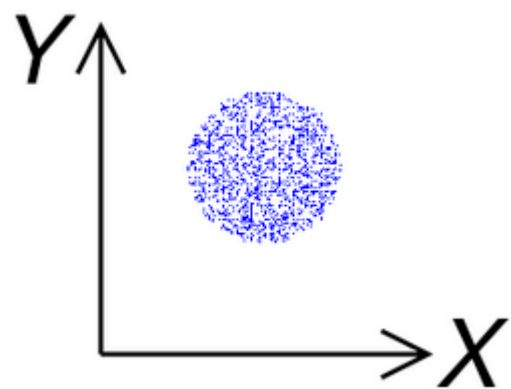
$$\mathbf{x}^{(i)} \in \mathbb{R}^n \quad \longrightarrow \quad \mathbf{c}^{(i)} \in \mathbb{R}^l \quad \longrightarrow \quad \hat{\mathbf{x}}^i \approx \mathbf{x}^i \in \mathbb{R}^n$$



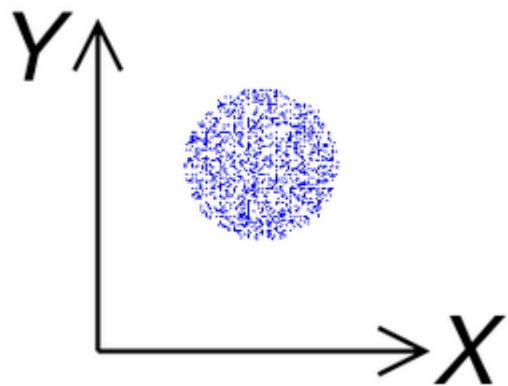
Suppose we have a collection of m points $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ in $\mathbb{R}^n$. Suppose we would like to apply lossy compression to these points. Lossy compression means storing the points in a way that requires less memory but may lose some precision. We would like to lose as little precision as possible.

$$\mathbf{x}^{(i)} \in \mathbb{R}^n \quad \longrightarrow \quad \mathbf{c}^{(i)} \in \mathbb{R}^l \quad \longrightarrow \quad \hat{\mathbf{x}}^i \approx \mathbf{x}^i \in \mathbb{R}^n$$

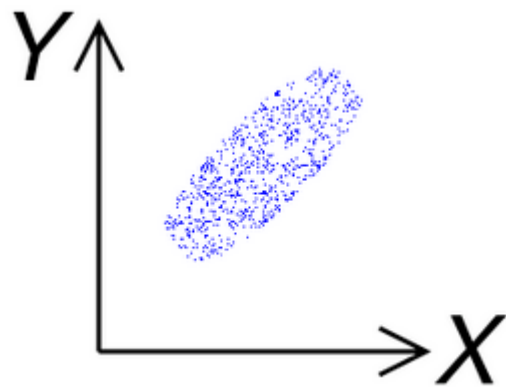




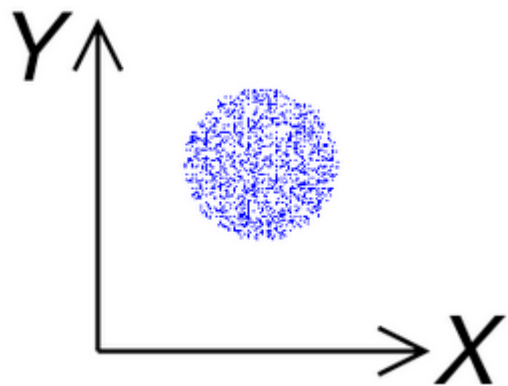
$$\text{cov}(X, Y) \approx 0$$



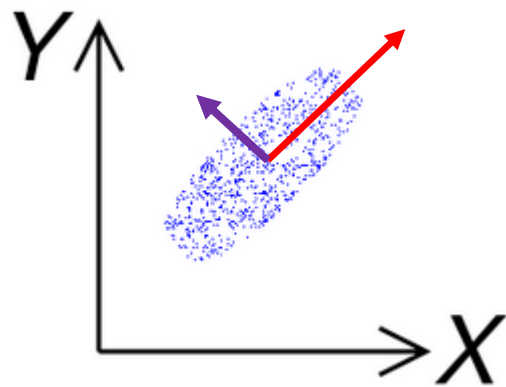
$$\text{cov}(X, Y) \approx 0$$



$$\text{cov}(X, Y) \neq 0$$



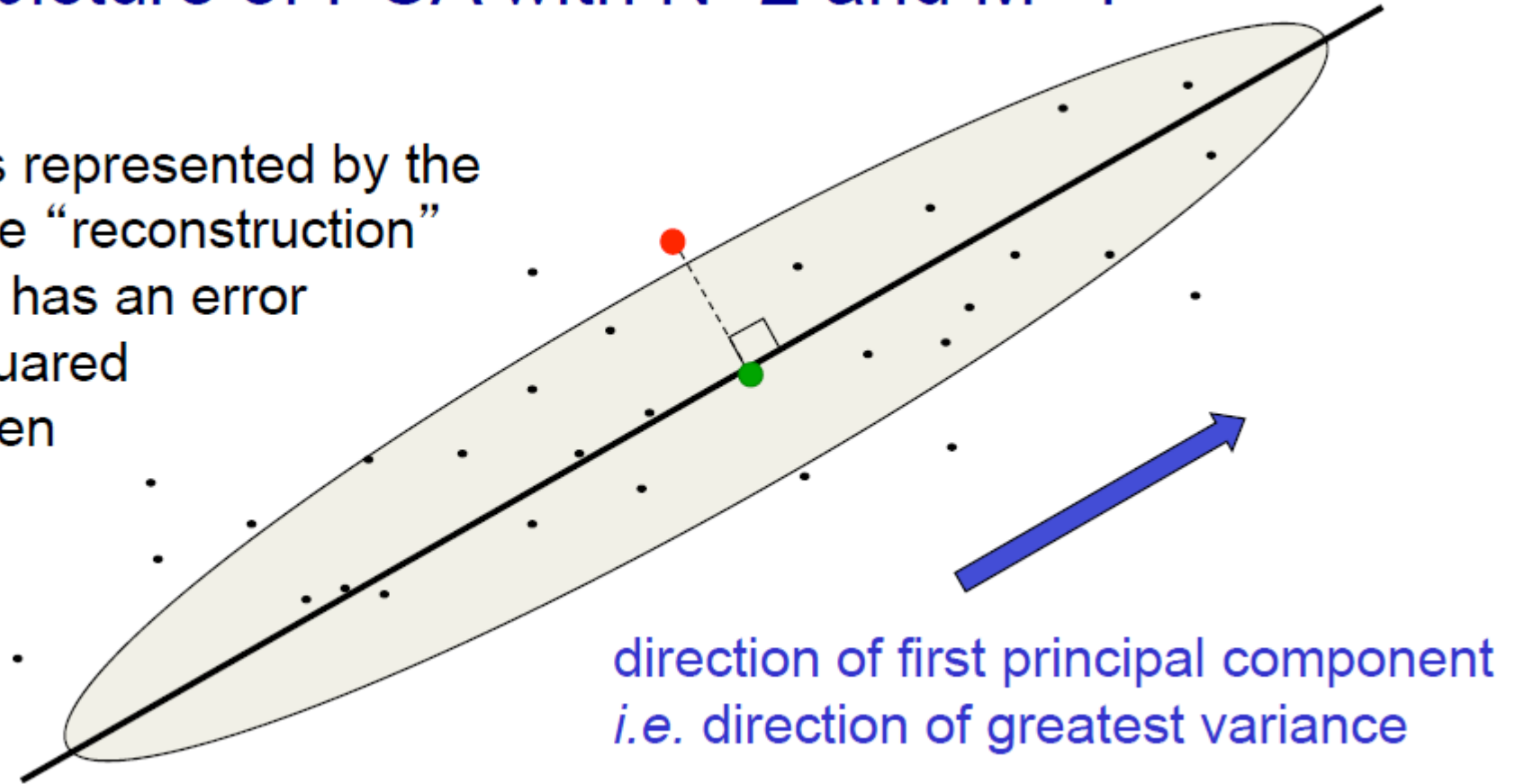
$$\text{cov}(X, Y) \approx 0$$

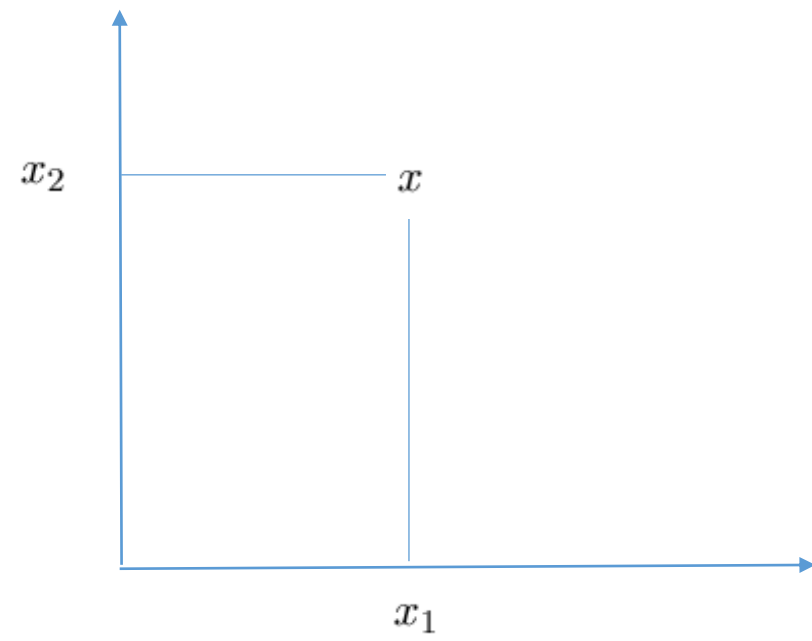


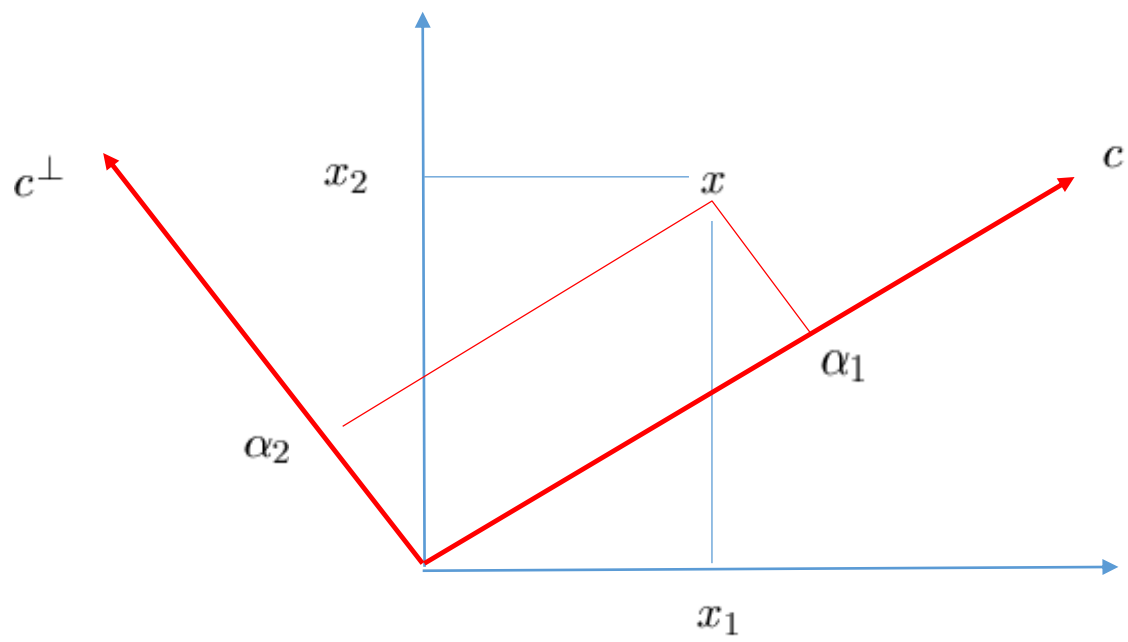
$$\text{cov}(X, Y) \neq 0$$

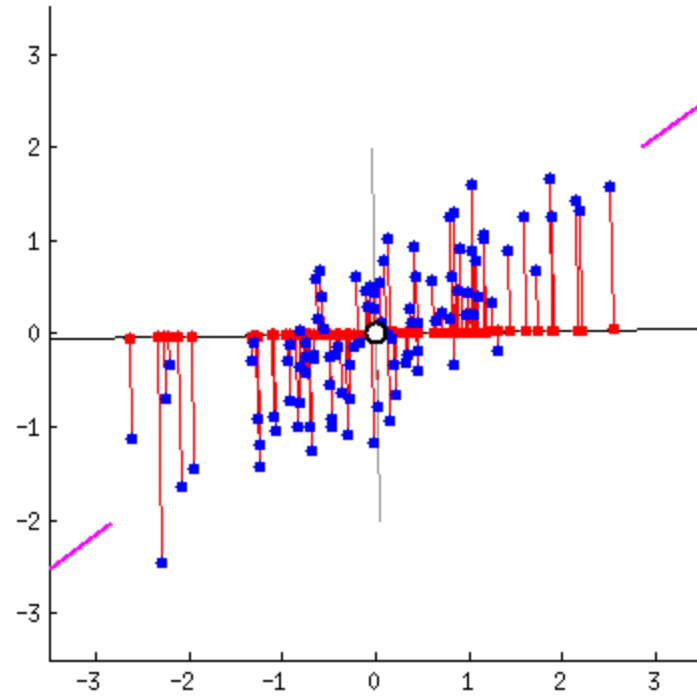
A picture of PCA with $N=2$ and $M=1$

The red point is represented by the green point. The “reconstruction” of the red point has an error equal to the squared distance between red and green points.









<https://builtin.com/data-science/step-step-explanation-principal-component-analysis>

$$X = [x^1 \ x^2 \ \dots x^m] \in \mathbf{R}^{n \times m}$$

Problem : Find a unit vector $c \in \mathbf{R}^n$ such that $\|c^T X\|_2$ is maximum.

$$X = [x^1 \ x^2 \ \dots x^m] \in \mathbf{R}^{n \times m}$$

Problem : Find a unit vector $c \in \mathbf{R}^n$ such that $\|c^T X\|_2$ is maximum.

$$c = \arg \max \frac{c^T X X^T c}{c^T c}$$

 Rayleigh Quotient

https://en.wikipedia.org/wiki/John_William_Strutt,_3rd_Baron_Rayleigh

$$X = [x^1 \ x^2 \ \dots x^m] \in \mathbf{R}^{n \times m}$$

Problem : Find a unit vector $c \in \mathbf{R}^n$ such that $\|c^T X\|_2$ is maximum.

$$c = \arg \max \frac{c^T X X^T c}{c^T c}$$

$$\max \frac{c^T X X^T c}{c^T c} = \lambda_{max}(X X^T)$$

c is the corresponding eigenvector

$$X = [x^1 \ x^2 \ \dots x^m] \in \mathbf{R}^{n \times m}$$

Problem : Find a unit vector $c \in \mathbf{R}^n$ such that $\|c^T X\|_2$ is maximum.

$$c = \arg \max \frac{c^T X X^T c}{c^T c}$$

$$\max \frac{c^T X X^T c}{c^T c} = \lambda_{max}(X X^T)$$

c is the corresponding eigenvalue

CALLED THE FIRST PRINCIPAL COMPONENT

Eigenvalues of XX^T : $\lambda_1 \geq \lambda_2 \geq \dots \lambda_n \geq 0$

Eigenvectors u_1 u_2 $\dots u_n$

Eigenvalues of XX^T : $\lambda_1 \geq \lambda_2 \geq \dots \lambda_n \geq 0$

Eigenvectors u_1 u_2 $\dots$ u_n



FIRST PRINCIPAL COMPONENT

Eigenvalues of XX^T : $\lambda_1 \geq \lambda_2 \geq \dots \lambda_n \geq 0$

Eigenvectors u_1 u_2 $\dots u_n$



SECOND PRINCIPAL COMPONENT

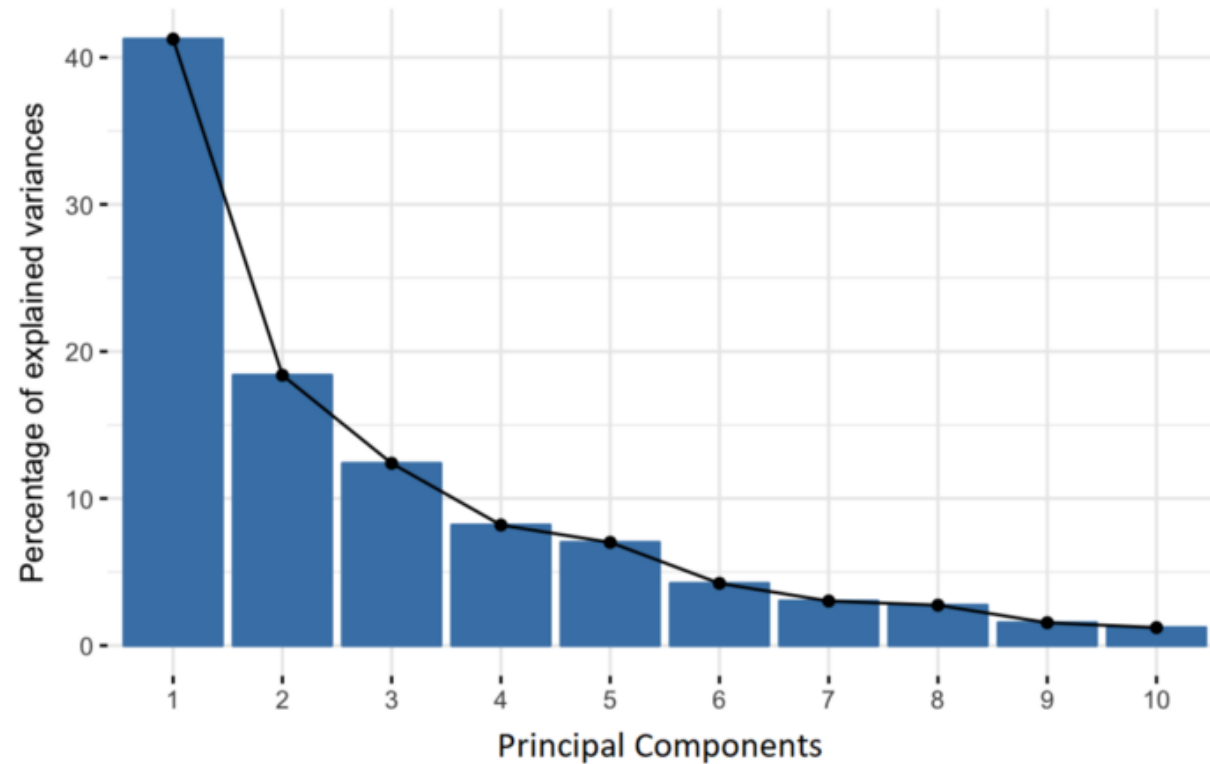
Eigenvalues of XX^T : $\lambda_1 \geq \lambda_2 \geq \dots \lambda_n \geq 0$

Eigenvectors u_1 u_2 $\dots$ u_n



nth PRINCIPAL COMPONENT

Principal components are new variables that are constructed as linear combinations or mixtures of the initial variables. These combinations are done in such a way that the new variables (i.e., principal components) are uncorrelated and most of the information within the initial variables is squeezed or compressed into the first components. So, the idea is 10-dimensional data gives you 10 principal components, but PCA tries to put maximum possible information in the first component, then maximum remaining information in the second and so on, until having something like shown in the scree plot below.



Eigenvalues of XX^T : $\lambda_1 \geq \lambda_2 \geq \dots \lambda_n \geq 0$

Eigenvectors $u_1 \quad u_2 \quad \dots u_n$

$$U = [u_1 \ u_2 \ u_n]$$

$$\Lambda = \text{diag}(\lambda_1, \lambda_2 \dots \lambda_n)$$

$$XX^T = U\Lambda U^T$$

$$x = \alpha_1 u_1 + \alpha_2 u_2 + \dots + \alpha_n u_n$$

$$x = \alpha_1 u_1 + \alpha_2 u_2 + \dots + \alpha_n u_n$$

$$\alpha_1 = u_1^T x$$

$$\alpha_i = u_i^T x$$

$$\alpha_n = u_n^T x$$

$$x = \alpha_1 u_1 + \alpha_2 u_2 + \dots + \alpha_n u_n$$

$$\alpha_1 = u_1^T x$$

$$\alpha_i = u_i^T x$$

$$\alpha_n = u_n^T x$$



$$\alpha = U^T x$$

$$x = \alpha_1 u_1 + \alpha_2 u_2 + \dots + \alpha_n u_n$$

$$\alpha_1 = u_1^T x$$

$$\alpha_i = u_i^T x$$

$$\alpha_n = u_n^T x$$



$$\alpha = U^T x$$

How to get reduced order representation ?

Singular Value Decomposition

$$X = U\Sigma V^T$$

[https://en.wikipedia.org/wiki/
Roger_Penrose](https://en.wikipedia.org/wiki/Roger_Penrose)

Singular Value Decomposition

$$X = U\Sigma V^T$$

$$\alpha(X) = U^T X = U^T U \Sigma V^T = \Sigma V^T$$

Singular Value Decomposition

$$X = U\Sigma V^T$$

$$\alpha(X) = U^T X = U^T U \Sigma V^T = \Sigma V^T$$

$$x = x^i = X e_i$$

Singular Value Decomposition

$$X = U\Sigma V^T$$

$$\alpha(X) = U^T X = U^T U \Sigma V^T = \Sigma V^T$$

$$x = x^i = X e_i$$



$$\alpha(x_i) = U^T X e_i = \Sigma V^T e_i = \begin{pmatrix} \sigma_1 * \\ \sigma_2 * \\ \sigma_k * \\ \sigma_{k+1} * \\ \vdots \\ \sigma_n * \end{pmatrix}$$

Singular Value Decomposition

$$X = U\Sigma V^T$$

$$\alpha(X) = U^T X = U^T U \Sigma V^T = \Sigma V^T$$

$$x = x^i = X e_i$$



$$\alpha(x_i) = U^T X e_i = \Sigma V^T e_i = \begin{pmatrix} \sigma_1 * \\ \sigma_2 * \\ \sigma_k * \\ \sigma_{k+1} * \\ \vdots \\ \sigma_n * \end{pmatrix}$$

Assume that $\sigma_{k+1} \approx 0$

Singular Value Decomposition

$$X = U\Sigma V^T$$

$$\alpha(X) = U^T X = U^T U \Sigma V^T = \Sigma V^T$$

$$x = x^i = X e_i$$



$$\alpha(x_i) = U^T X e_i = \Sigma V^T e_i = \begin{pmatrix} \sigma_1 * \\ \sigma_2 * \\ \sigma_k * \\ \sigma_{k+1} * \\ \vdots \\ \sigma_n * \end{pmatrix} \Bigg] \approx 0$$

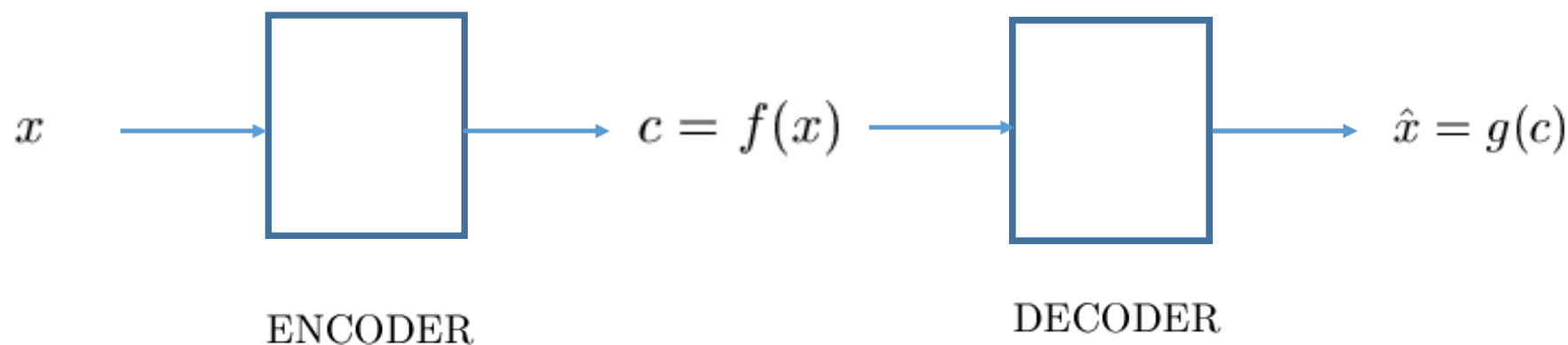
Assume that $\sigma_{k+1} \approx 0$

$$x = x^i = X e_i \in \mathbf{R}^n \quad \longrightarrow \quad \alpha(x_i) = \begin{pmatrix} \sigma_1 * \\ \sigma_2 * \\ \sigma_k * \\ 0 \\ \vdots \\ 0 \end{pmatrix} \quad \longrightarrow \quad c(x_i) = \begin{pmatrix} \sigma_1 * \\ \sigma_2 * \\ \sigma_k * \end{pmatrix} \in \mathbf{R}^k$$

$$c(x_i) = (U_{:,1:k})^T x_i$$

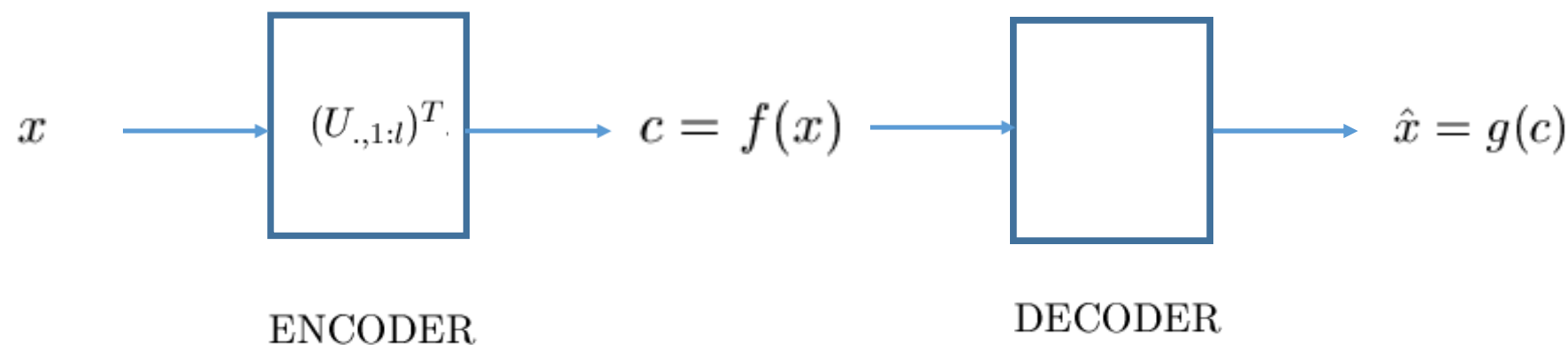
Suppose we have a collection of m points $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ in $\mathbb{R}^n$. Suppose we would like to apply lossy compression to these points. Lossy compression means storing the points in a way that requires less memory but may lose some precision. We would like to lose as little precision as possible.

$$\mathbf{x}^{(i)} \in \mathbb{R}^n \quad \longrightarrow \quad \mathbf{c}^{(i)} \in \mathbb{R}^l \quad \longrightarrow \quad \hat{\mathbf{x}}^i \approx \mathbf{x}^i \in \mathbb{R}^n$$



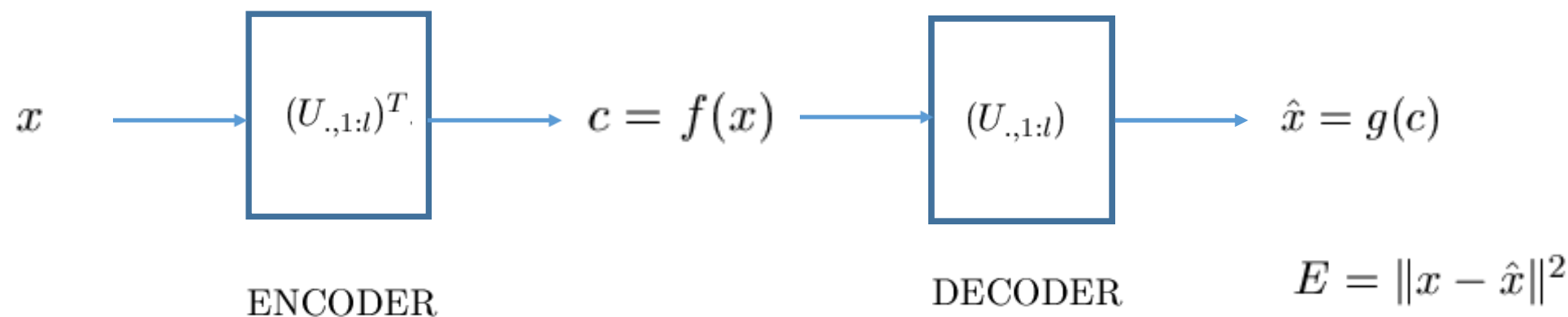
Suppose we have a collection of m points $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ in $\mathbb{R}^n$. Suppose we would like to apply lossy compression to these points. Lossy compression means storing the points in a way that requires less memory but may lose some precision. We would like to lose as little precision as possible.

$$\mathbf{x}^{(i)} \in \mathbb{R}^n \quad \longrightarrow \quad \mathbf{c}^{(i)} \in \mathbb{R}^l \quad \longrightarrow \quad \hat{\mathbf{x}}^i \approx \mathbf{x}^i \in \mathbb{R}^n$$



Suppose we have a collection of m points $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ in $\mathbb{R}^n$. Suppose we would like to apply lossy compression to these points. Lossy compression means storing the points in a way that requires less memory but may lose some precision. We would like to lose as little precision as possible.

$$\mathbf{x}^{(i)} \in \mathbb{R}^n \quad \longrightarrow \quad \mathbf{c}^{(i)} \in \mathbb{R}^l \quad \longrightarrow \quad \hat{\mathbf{x}}^i \approx \mathbf{x}^i \in \mathbb{R}^n$$



$$X = U\Sigma V^T$$

$$X = U\Sigma V^T$$

$$U = [u_1 \ u_2 \ \dots u_k \ u_{k+1} \ \dots u_n]$$

$$V = [v_1 \ v_2 \ \dots v_k \ v_{k+1} \ \dots v_m]$$

$$X = U\Sigma V^T$$

$$U = [u_1 \ u_2 \ \dots u_k \ u_{k+1} \ \dots u_n]$$

$$V = [v_1 \ v_2 \ \dots v_k \ v_{k+1} \ \dots v_m]$$

$$X = u_1\sigma_1v_1^T + \dots + u_k\sigma_kv_k^T + \sigma_{k+1}u_{k+1}\sigma_{k+1}v_{k+1}^T + \dots + u_n\sigma_nv_n^T$$

$$X = U\Sigma V^T$$

$$U = [u_1 \ u_2 \ \dots u_k \ u_{k+1} \ \dots u_n]$$

$$V = [v_1 \ v_2 \ \dots v_k \ v_{k+1} \ \dots v_m]$$

$$X = u_1 \sigma_1 v_1^T + \dots + u_k \sigma_k v_k^T + \sigma_{k+1} u_{k+1} \sigma_{k+1} v_{k+1}^T + \dots + u_n \sigma_n v_n^T$$

$$x_i = X e_i = u_1 \sigma_1 v_1^T e_i + \dots + \underbrace{u_k \sigma_k v_k^T}_{\alpha_k} e_i + u_{k+1} \sigma_{k+1} v_{k+1}^T e_i + \dots + u_n \sigma_n v_n^T e_i$$

$$X = U\Sigma V^T$$

$$U = [u_1 \ u_2 \ \dots u_k \ u_{k+1} \ \dots u_n]$$

$$V = [v_1 \ v_2 \ \dots v_k \ v_{k+1} \ \dots v_m]$$

$$X = u_1\sigma_1v_1^T + \dots + u_k\sigma_kv_k^T + \sigma_{k+1}u_{k+1}\sigma_{k+1}v_{k+1}^T + \dots + u_n\sigma_nv_n^T$$

$$x_i = Xe_i = u_1\sigma_1v_1^Te_i + \dots + u_k\sigma_kv_k^Te_i + u_{k+1}\sigma_{k+1}v_{k+1}^Te_i + \dots + u_n\sigma_nv_n^Te_i$$

$$\underbrace{\hspace{10em}}_{\hat{x}}$$

$$X = U\Sigma V^T$$

$$U = [u_1 \ u_2 \ \dots u_k \ u_{k+1} \ \dots u_n]$$

$$V = [v_1 \ v_2 \ \dots v_k \ v_{k+1} \ \dots v_m]$$

$$X = u_1 \sigma_1 v_1^T + \dots + u_k \sigma_k v_k^T + \sigma_{k+1} u_{k+1} \sigma_{k+1} v_{k+1}^T + \dots + u_n \sigma_n v_n^T$$

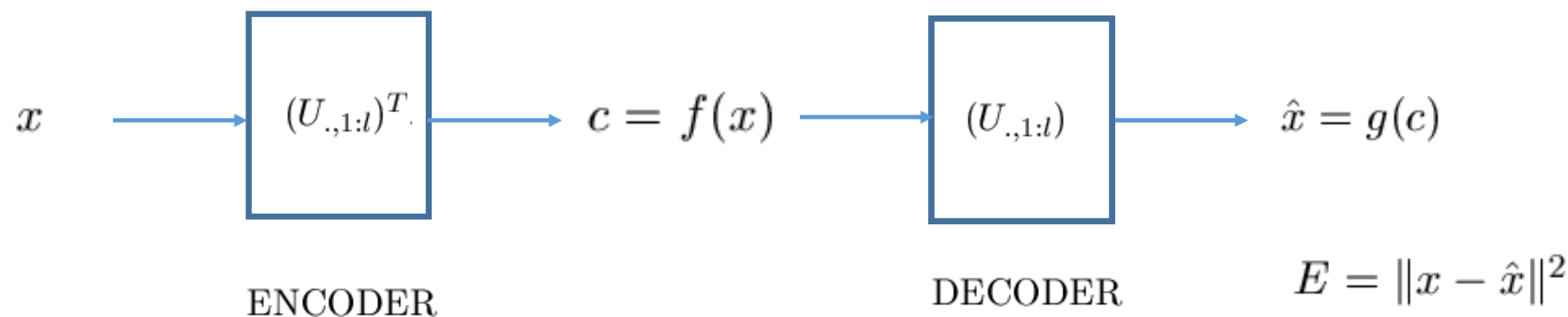
$$x_i = X e_i = u_1 \sigma_1 v_1^T e_i + \dots + u_k \sigma_k v_k^T e_i + u_{k+1} \sigma_{k+1} v_{k+1}^T e_i + \dots + u_n \sigma_n v_n^T e_i$$

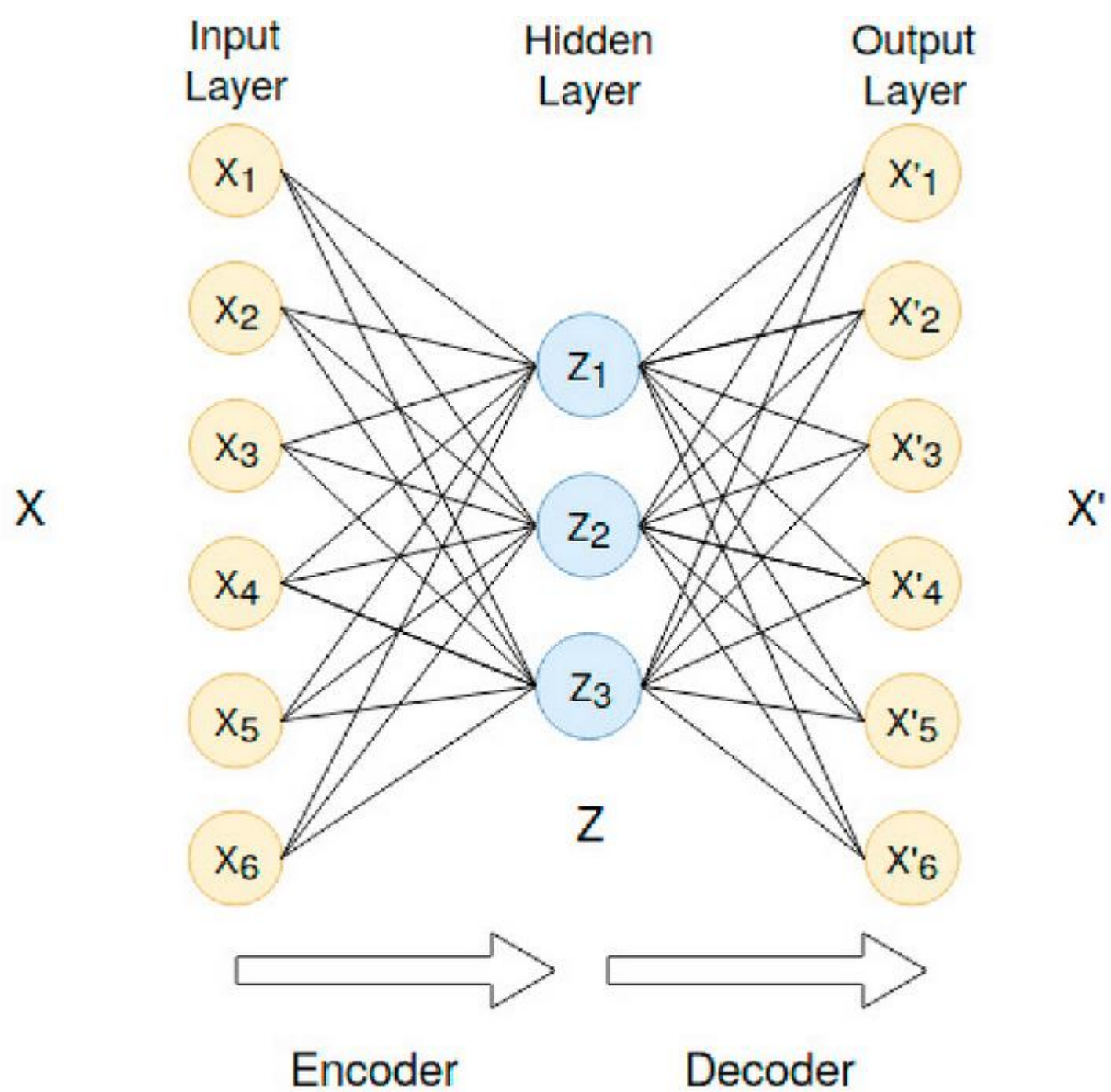


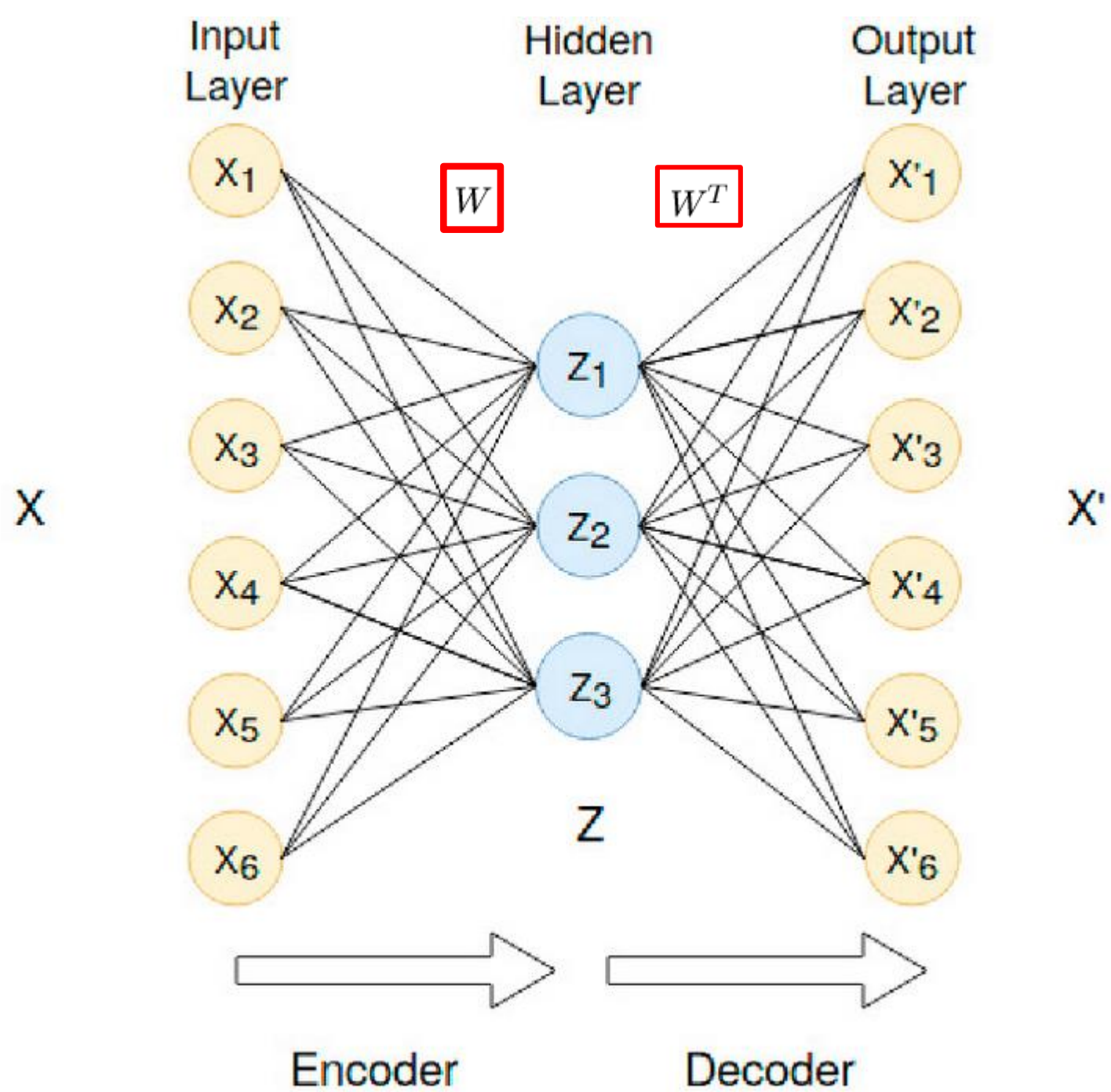
$$e = x - \hat{x}$$

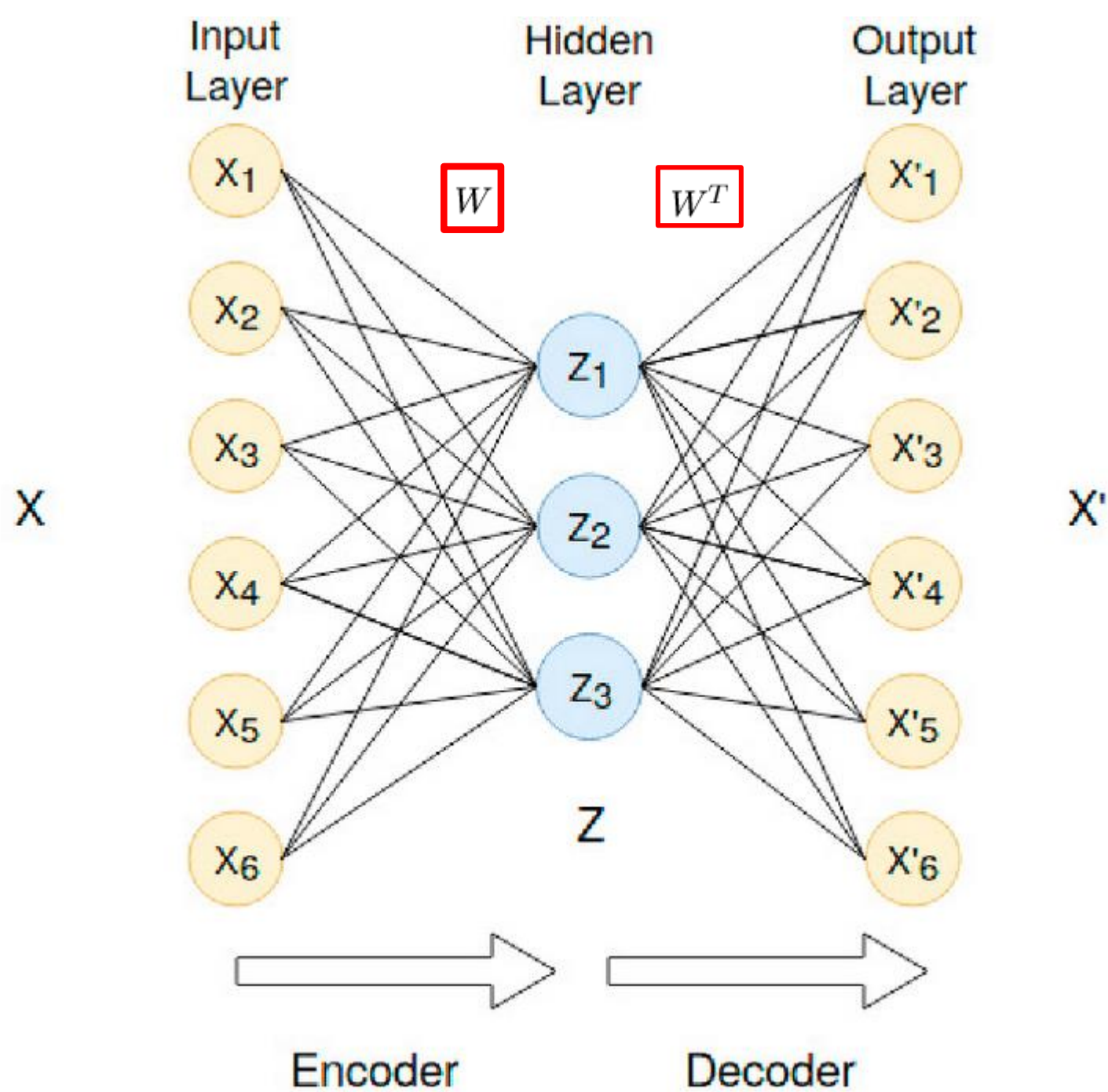
Suppose we have a collection of m points $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ in $\mathbb{R}^n$. Suppose we would like to apply lossy compression to these points. Lossy compression means storing the points in a way that requires less memory but may lose some precision. We would like to lose as little precision as possible.

$$\mathbf{x}^{(i)} \in \mathbb{R}^n \quad \longrightarrow \quad \mathbf{c}^{(i)} \in \mathbb{R}^l \quad \longrightarrow \quad \hat{\mathbf{x}}^i \approx \mathbf{x}^i \in \mathbb{R}^n$$









$$E = \|x - \hat{x}\|^2$$

UNSUPERVISED
LEARNING

Backpropagation with weight constraints

- It's easy to modify the backpropagation algorithm to incorporate linear constraints between the weights.
- We compute the gradients as usual, and then modify the gradients so that they satisfy the constraints.
 - So if the weights started off satisfying the constraints, they will continue to satisfy them.

*To constrain: $w_1 = w_2$
we need: $\Delta w_1 = \Delta w_2$*

compute: $\frac{\partial E}{\partial w_1}$ and $\frac{\partial E}{\partial w_2}$

use $\frac{\partial E}{\partial w_1} + \frac{\partial E}{\partial w_2}$ for w_1 and w_2

UNSUPERVISED LEARNING

Topics: unsupervised learning

- Unsupervised learning: only use the inputs $\mathbf{x}^{(t)}$ for learning
 - automatically extract meaningful features for your data
 - leverage the availability of unlabeled data
 - add a data-dependent regularizer to trainings

UNSUPERVISED LEARNING

Topics: unsupervised learning

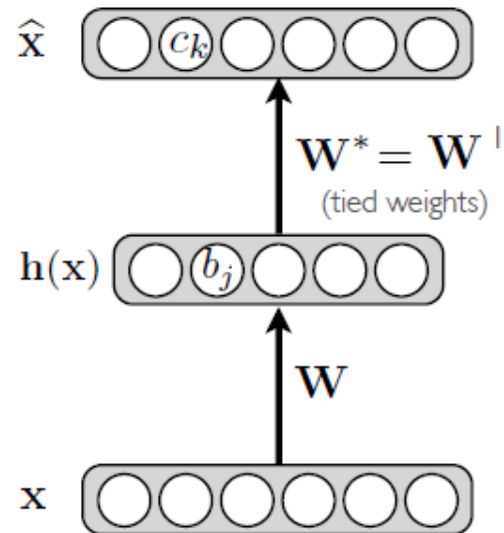
- Unsupervised learning: only use the inputs $\mathbf{x}^{(t)}$ for learning
 - automatically extract meaningful features for your data
 - leverage the availability of unlabeled data
 - add a data-dependent regularizer to trainings
- One of 3 common neural networks for unsupervised learning
 - restricted Boltzmann machines
 - **autoencoders**
 - sparse coding model

https://info.usherbrooke.ca/hlarochelle/neural_networks/content.html

AUTOENCODER

Topics: autoencoder, encoder, decoder, tied weights

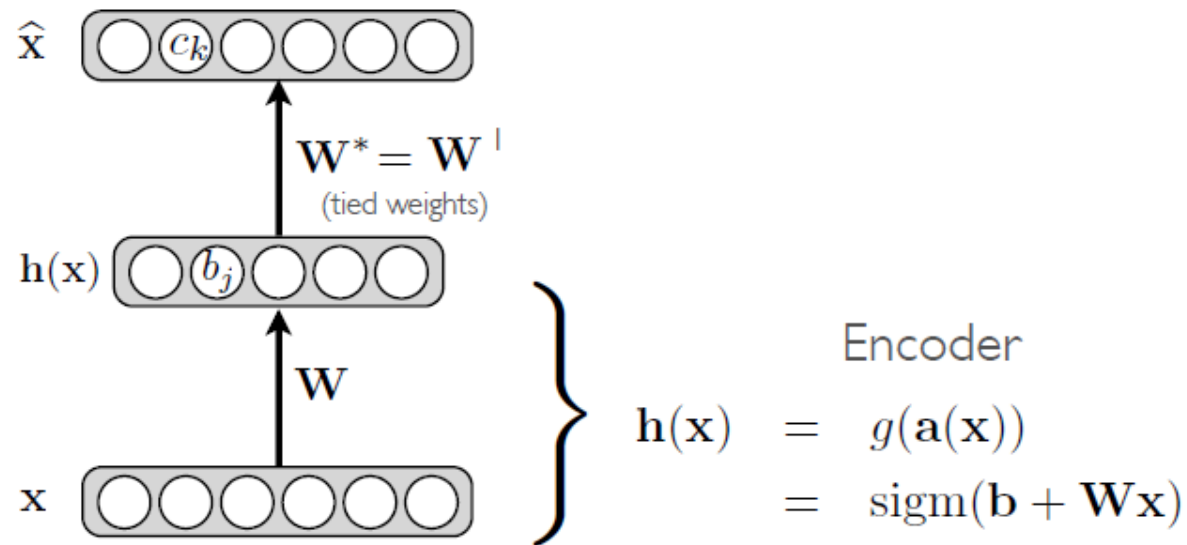
- Feed-forward neural network trained to reproduce its input at the output layer



AUTOENCODER

Topics: autoencoder, encoder, decoder, tied weights

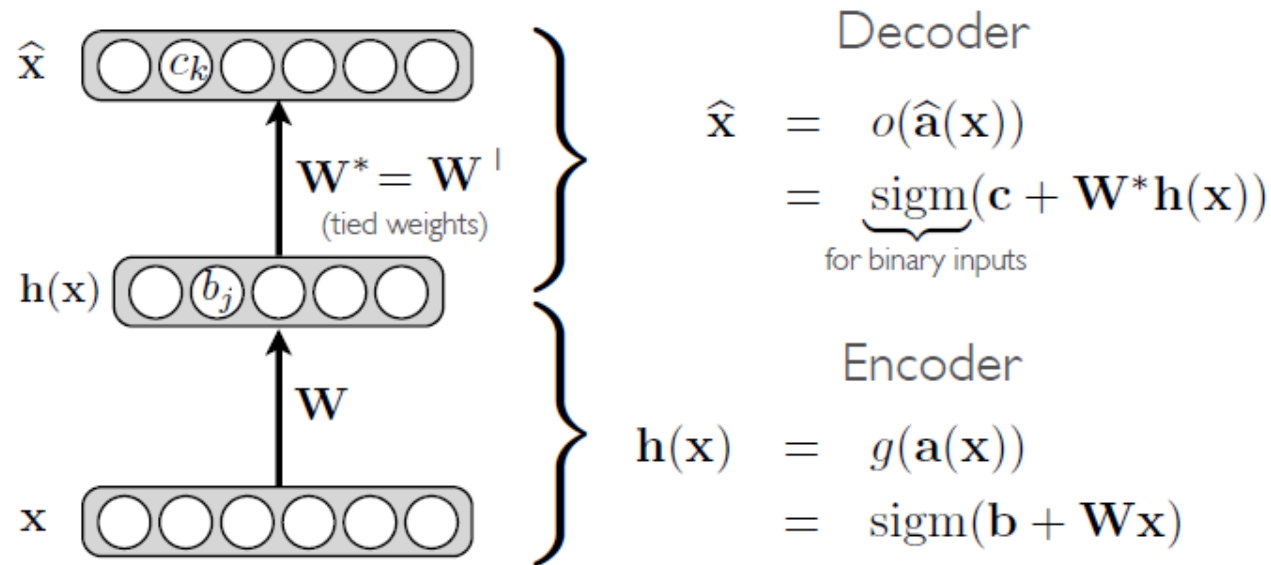
- Feed-forward neural network trained to reproduce its input at the output layer



AUTOENCODER

Topics: autoencoder, encoder, decoder, tied weights

- Feed-forward neural network trained to reproduce its input at the output layer



AUTOENCODER

Topics: loss function

AUTOENCODER

Topics: loss function

- For binary inputs:

$$f(\mathbf{x}) \equiv \hat{\mathbf{x}}$$

$$l(f(\mathbf{x})) = - \sum_k (x_k \log(\hat{x}_k) + (1 - x_k) \log(1 - \hat{x}_k))$$

- cross-entropy (more precisely: sum of Bernoulli cross-entropies)

AUTOENCODER

Topics: loss function

- For binary inputs:

$$f(\mathbf{x}) \equiv \hat{\mathbf{x}}$$

$$l(f(\mathbf{x})) = - \sum_k (x_k \log(\hat{x}_k) + (1 - x_k) \log(1 - \hat{x}_k))$$

- cross-entropy (more precisely: sum of Bernoulli cross-entropies)

- For real-valued inputs:

$$l(f(\mathbf{x})) = \frac{1}{2} \sum_k (\hat{x}_k - x_k)^2$$

- sum of squared differences (squared euclidean distance)
- we use a linear activation function at the output

AUTOENCODER

Topics: loss function gradient

- For both cases, the gradient $\nabla_{\hat{\mathbf{a}}(\mathbf{x}^{(t)})} l(f(\mathbf{x}^{(t)}))$ has a very simple form:

$$f(\mathbf{x}) \equiv \hat{\mathbf{x}}$$

AUTOENCODER

Topics: loss function gradient

- For both cases, the gradient $\nabla_{\hat{\mathbf{a}}(\mathbf{x}^{(t)})} l(f(\mathbf{x}^{(t)}))$ has a very simple form:

$$f(\mathbf{x}) \equiv \hat{\mathbf{x}}$$

$$\nabla_{\hat{\mathbf{a}}(\mathbf{x}^{(t)})} l(f(\mathbf{x}^{(t)})) = \hat{\mathbf{x}}^{(t)} - \mathbf{x}^{(t)}$$

AUTOENCODER

Topics: loss function gradient

- For both cases, the gradient $\nabla_{\hat{\mathbf{a}}(\mathbf{x}^{(t)})} l(f(\mathbf{x}^{(t)}))$ has a very simple form:

$$f(\mathbf{x}) \equiv \hat{\mathbf{x}}$$

$$\nabla_{\hat{\mathbf{a}}(\mathbf{x}^{(t)})} l(f(\mathbf{x}^{(t)})) = \hat{\mathbf{x}}^{(t)} - \mathbf{x}^{(t)}$$

- Parameter gradients are obtained by backpropagating the gradient $\nabla_{\hat{\mathbf{a}}(\mathbf{x}^{(t)})} l(f(\mathbf{x}^{(t)}))$ like in a regular network

AUTOENCODER

Topics: loss function gradient

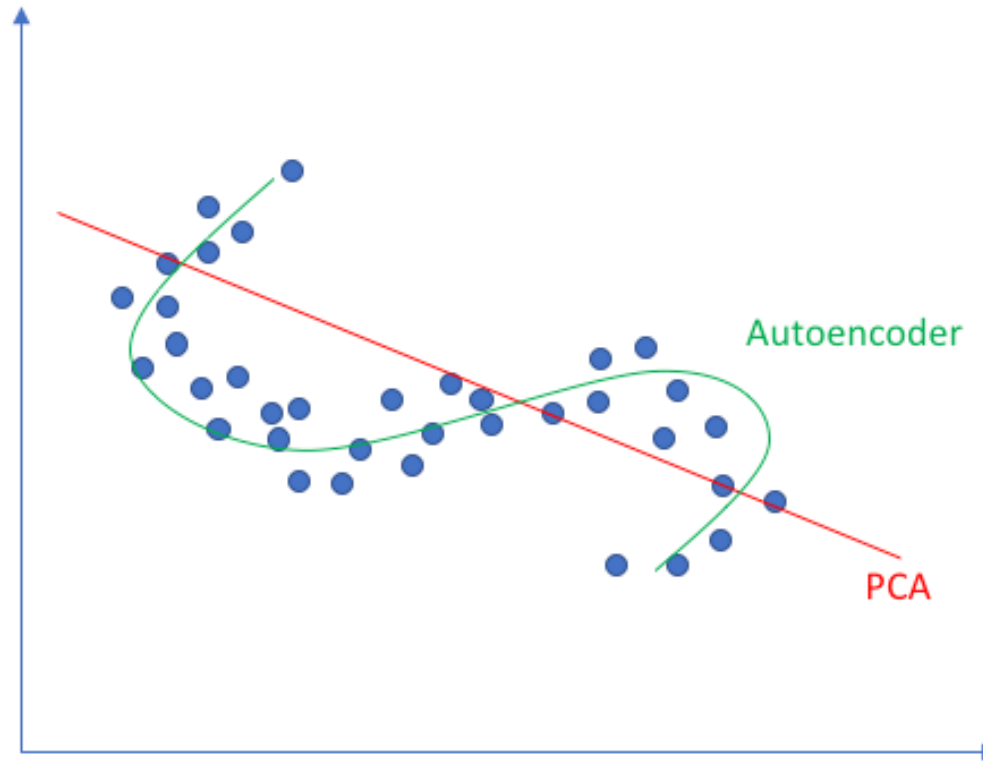
- For both cases, the gradient $\nabla_{\hat{\mathbf{a}}(\mathbf{x}^{(t)})} l(f(\mathbf{x}^{(t)}))$ has a very simple form:

$$f(\mathbf{x}) \equiv \hat{\mathbf{x}}$$

$$\nabla_{\hat{\mathbf{a}}(\mathbf{x}^{(t)})} l(f(\mathbf{x}^{(t)})) = \hat{\mathbf{x}}^{(t)} - \mathbf{x}^{(t)}$$

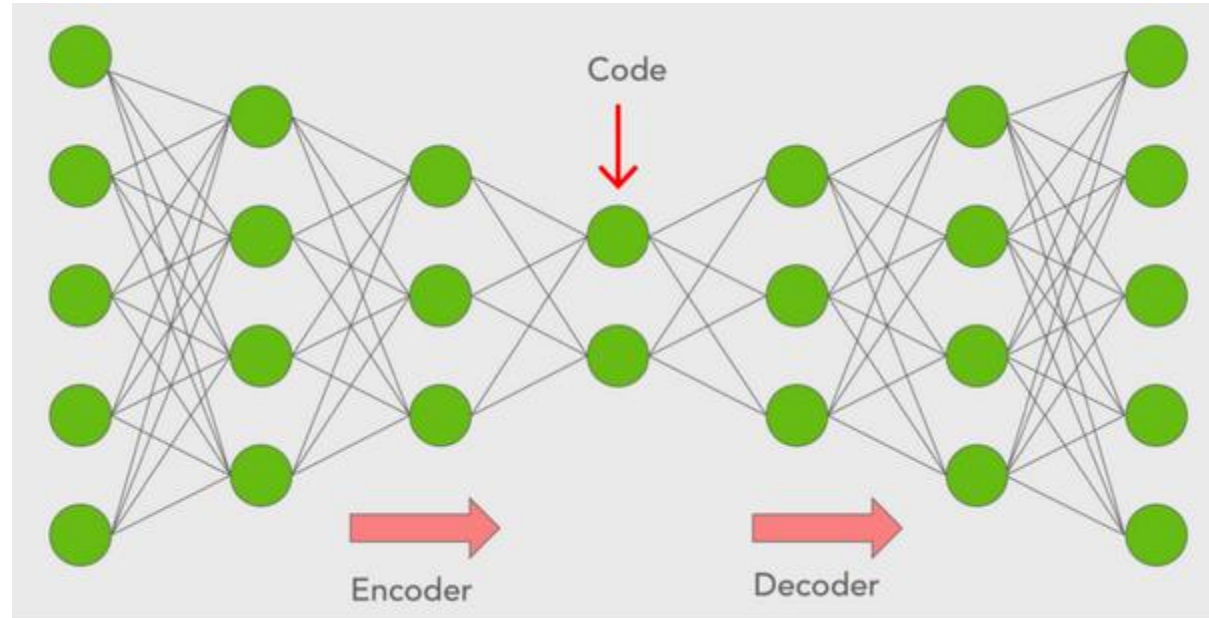
- Parameter gradients are obtained by backpropagating the gradient $\nabla_{\hat{\mathbf{a}}(\mathbf{x}^{(t)})} l(f(\mathbf{x}^{(t)}))$ like in a regular network
 - **important:** when using tied weights ($\mathbf{w}^* = \mathbf{w}^\top$), $\nabla_{\mathbf{w}} l(f(\mathbf{x}^{(t)}))$ is the sum of two gradients !
 - this is because $\mathbf{W}$ is present in the encoder **and** in the decoder

Linear vs nonlinear dimensionality reduction



<https://www.jeremyjordan.me/autoencoders/>

For higher dimensional data, autoencoders are capable of learning a complex representation of the data (manifold) which can be used to describe observations in a lower dimensionality and correspondingly decoded into the original input space.



blue bars in Fig. 1 summarize the measured SHG signals. For excitation of the *LC* resonance in Fig. 1A (horizontal incident polarization), we find an SHG signal that is 500 times above the noise level. As expected for SHG, this signal closely scales with the square of the incident power (Fig. 2A). The polarization of the SHG emission is nearly vertical (Fig. 2B). The small angle with respect to the vertical is due to deviations from perfect mirror symmetry of the SRRs (see electron micrographs in Fig. 1). Small detuning of the *LC* resonance toward smaller wavelength (i.e., to 1.3- μ m wavelength) reduces the SHG signal strength from 100% to 20%. For excitation of the Mie resonance with vertical incident polarization in Fig. 1D, we find a small signal just above the noise level. For excitation of the Mie resonance with horizontal incident polarization in Fig. 1C, a small but significant SHG emission is found, which is again po-

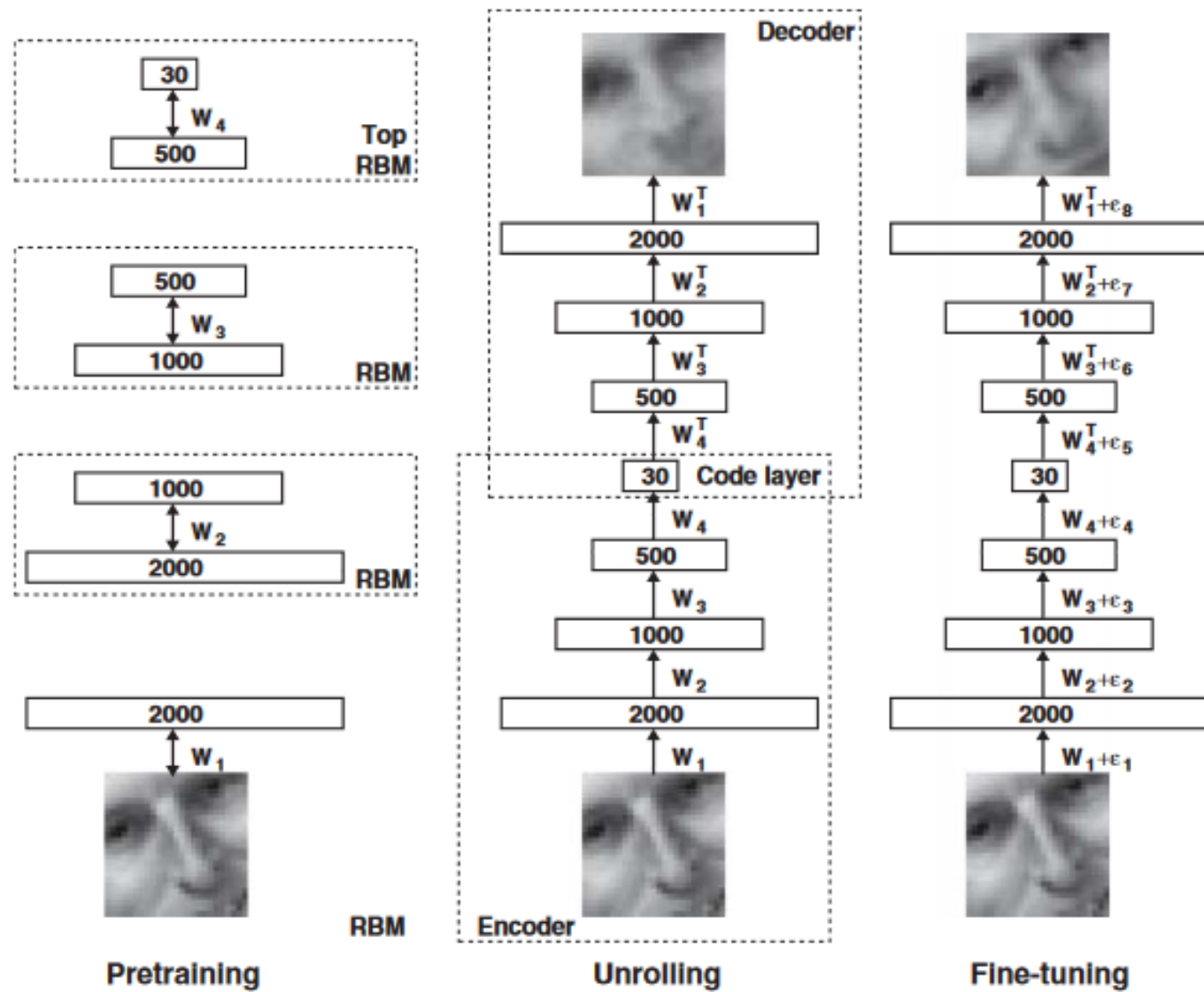
Reducing the Dimensionality of Data with Neural Networks

G. E. Hinton* and R. R. Salakhutdinov

High-dimensional data can be converted to low-dimensional codes by training a multilayer neural network with a small central layer to reconstruct high-dimensional input vectors. Gradient descent can be used for fine-tuning the weights in such “autoencoder” networks, but this works well only if the initial weights are close to a good solution. We describe an effective way of initializing the weights that allows deep autoencoder networks to learn low-dimensional codes that work much better than principal components analysis as a tool to reduce the dimensionality of data.

Dimensionality reduction facilitates the classification, visualization, communication, and storage of high-dimensional data. A simple and widely used method is principal components analysis (PCA), which

finds the directions of greatest variance in the data set and represents each data point by its coordinates along each of these directions. We describe a nonlinear generalization of PCA that uses an adaptive, multilayer “encoder” network





1.44, 7.64, 2.45, 5.90. (B) Top to bottom: A random test image from each class; reconstructions by the 30-dimensional autoencoder; reconstructions by 30-dimensional logistic PCA and standard PCA. The average squared errors for the last three rows are 3.00, 8.01, and 13.87.

Using Very Deep Autoencoders for Content-Based Image Retrieval

Alex Krizhevsky and Geoffrey E. Hinton

University of Toronto - Department of Computer Science
6 King's College Road, Toronto, M5S 3H5 - Canada

Abstract. We show how to learn many layers of features on color images and we use these features to initialize deep autoencoders. We then use the autoencoders to map images to short binary codes. Using semantic hashing [6], 28-bit codes can be used to retrieve images that are similar to a query image in a time that is independent of the size of the database. This extremely fast retrieval makes it possible to search using multiple different transformations of the query image. 256-bit binary codes allow much more accurate matching and can be used to prune the set of images found using the 28-bit codes.

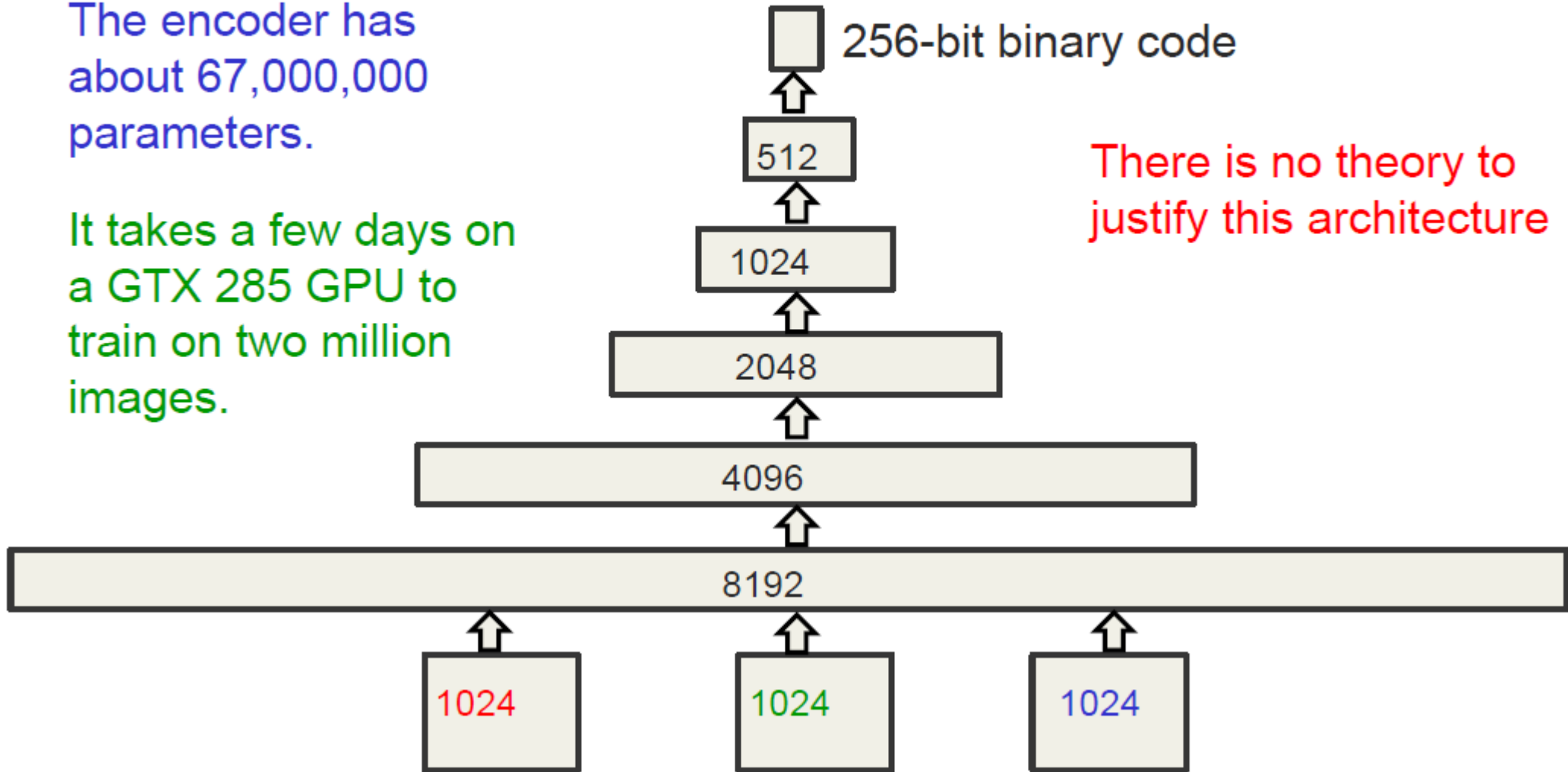
<https://www.cs.utoronto.ca/~hinton/absps/esann-deep-final.pdf>

Krizhevsky's deep autoencoder

The encoder has about 67,000,000 parameters.

It takes a few days on a GTX 285 GPU to train on two million images.

There is no theory to justify this architecture



256-bit deep



Exploring Strategies for Training Deep Neural Networks

Hugo Larochelle

Yoshua Bengio

Jérôme Louradour

Pascal Lamblin

Département d'informatique et de recherche opérationnelle

Université de Montréal

2920, chemin de la Tour

Montréal, Québec, Canada, H3T 1J8

LAROCHEH@IRO.UMONTREAL.CA

BENGIOY@IRO.UMONTREAL.CA

LOURADOJ@IRO.UMONTREAL.CA

LAMBLINP@IRO.UMONTREAL.CA

Editor: Léon Bottou

Abstract

Deep multi-layer neural networks have many levels of non-linearities allowing them to compactly represent highly non-linear and highly-varying functions. However, until recently it was not clear how to train such deep networks, since gradient-based optimization starting from random initialization often appears to get stuck in poor solutions. Hinton et al. recently proposed a greedy layer-wise unsupervised learning procedure relying on the training algorithm of restricted Boltzmann machines (RBM) to initialize the parameters of a deep belief network (DBN), a generative model with many layers of hidden causal variables. This was followed by the proposal of another

<https://www.jmlr.org/papers/volume10/larochelle09a/larochelle09a.pdf>

3	8	6	9	6	4	5	3	8	4	5	2	3	8	4	8
1	5	0	5	9	7	4	1	0	3	0	6	2	9	9	4
1	3	6	8	0	7	7	6	8	9	0	3	8	3	7	7
8	4	4	1	2	9	8	1	1	0	6	6	5	0	1	1
7	2	7	3	1	4	0	5	0	6	8	7	6	8	9	9
4	0	6	1	9	2	2	3	9	4	4	5	6	6	1	7
2	8	6	9	7	0	9	1	6	2	8	3	6	4	9	5
8	6	8	7	8	8	6	9	1	7	6	0	9	6	7	0

Figure 5: Samples from the MNIST digit recognition data set. Here, a black pixel corresponds to an input value of 0 and a white pixel corresponds to 1 (the inputs are scaled between 0 and 1).

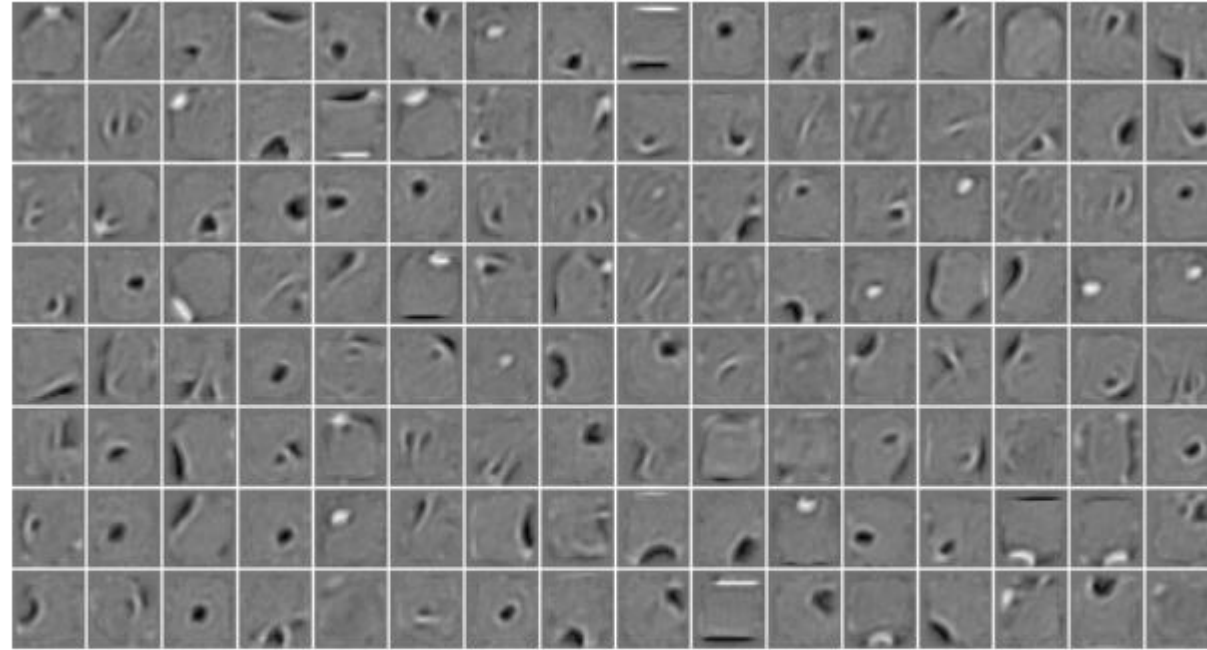


Figure 6: Display of the input weights of a random subset of the hidden units, learned by an RBM when trained on samples from the MNIST data set. The activation of units of the first

AUTOENCODER

Topics: optimality of a linear autoencoder

AUTOENCODER

Topics: optimality of a linear autoencoder

- To do the proof, we need the following theorem:
 - let $\mathbf{A}$ be any matrix, with singular value decomposition $\mathbf{A} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^T$
 - $\mathbf{\Sigma}$ is a diagonal matrix
 - $\mathbf{U}, \mathbf{V}$ are orthonormal matrices (columns/rows are orthonormal vectors)

AUTOENCODER

Topics: optimality of a linear autoencoder

- To do the proof, we need the following theorem:
 - let $\mathbf{A}$ be any matrix, with singular value decomposition $\mathbf{A} = \mathbf{U} \Sigma \mathbf{V}^\top$
 - Σ is a diagonal matrix
 - $\mathbf{U}, \mathbf{V}$ are orthonormal matrices (columns/rows are orthonormal vectors)
 - let $\mathbf{U}_{:, \leq k} \Sigma_{\leq k, \leq k} \mathbf{V}_{:, \leq k}^\top$ be the decomposition where we keep only the k largest singular values
 - then, the matrix $\mathbf{B}$ of rank k that is closest to $\mathbf{A}$:

$$\mathbf{B}^* = \arg \min_{\mathbf{B} \text{ s.t. rank}(\mathbf{B})=k} \|\mathbf{A} - \mathbf{B}\|_F$$

is $\mathbf{B}^* = \mathbf{U}_{:, \leq k} \Sigma_{\leq k, \leq k} \mathbf{V}_{:, \leq k}^\top$

$$\min_{\theta} \sum_t \frac{1}{2} \sum_i (x_i^{(t)} - \underbrace{\hat{x}_i^{(t)}}_{\substack{\text{based on} \\ \text{linear encoder}}})^2$$

Sketch of proof

$$\min_{\theta} \sum_t \frac{1}{2} \sum_i (x_i^{(t)} - \underbrace{\hat{x}_i^{(t)}}_{\substack{\text{based on} \\ \text{linear encoder}}})^2 = \min_{\mathbf{W}^*, \mathbf{h}(\mathbf{X})} \frac{1}{2} \|\underbrace{\mathbf{X}}_{\substack{\text{matrix where columns are } \mathbf{x}^{(t)} \\ \text{matrix of all hidden layers} \\ \text{(could be any encoder)}}} - \mathbf{W}^* \mathbf{h}(\mathbf{X})\|_F^2$$

Sketch of proof

$$\min_{\theta} \sum_t \frac{1}{2} \sum_i (x_i^{(t)} - \underbrace{\hat{x}_i^{(t)}}_{\substack{\text{based on} \\ \text{linear encoder}}})^2 = \min_{\mathbf{W}^*, \mathbf{h}(\mathbf{X})} \frac{1}{2} \|\underbrace{\mathbf{X}}_{\substack{\text{matrix where columns are } \mathbf{x}^{(t)} \\ \text{matrix of all hidden layers} \\ \text{(could be any encoder)}}} - \mathbf{W}^* \mathbf{h}(\mathbf{X})\|_F^2$$

$$\arg \min_{\mathbf{W}^*, \mathbf{h}(\mathbf{X})} \frac{1}{2} \|\mathbf{X} - \mathbf{W}^* \mathbf{h}(\mathbf{X})\|_F^2$$

Sketch of proof

$$\arg \min_{\mathbf{W}^*, \mathbf{h}(\mathbf{X})} \frac{1}{2} \|\mathbf{X} - \mathbf{W}^* \mathbf{h}(\mathbf{X})\|_F^2 = (\mathbf{W}^* \leftarrow \mathbf{U}_{:, \leq k} \Sigma_{\leq k, \leq k}, \mathbf{h}(\mathbf{X}) \leftarrow \mathbf{V}_{:, \leq k}^\top)$$

based on previous theorem, where $\mathbf{X} = \mathbf{U} \Sigma \mathbf{V}^\top$
and k is the hidden layer size

$$\arg \min_{\mathbf{W}^*, \mathbf{h}(\mathbf{X})} \frac{1}{2} \|\mathbf{X} - \mathbf{W}^* \mathbf{h}(\mathbf{X})\|_F^2 = (\mathbf{W}^* \leftarrow \mathbf{U}_{:, \leq k} \Sigma_{\leq k, \leq k}, \mathbf{h}(\mathbf{X}) \leftarrow \mathbf{V}_{:, \leq k}^\top)$$

based on previous theorem, where $\mathbf{X} = \mathbf{U} \Sigma \mathbf{V}^\top$
and k is the hidden layer size

Let's show $\mathbf{h}(\mathbf{X})$ is a linear encoder:

$$\mathbf{h}(\mathbf{X}) = \mathbf{V}_{:, \leq k}^\top$$

$$\arg \min_{\mathbf{W}^*, \mathbf{h}(\mathbf{X})} \frac{1}{2} \|\mathbf{X} - \mathbf{W}^* \mathbf{h}(\mathbf{X})\|_F^2 = (\mathbf{W}^* \leftarrow \mathbf{U}_{:, \leq k} \Sigma_{\leq k, \leq k}, \mathbf{h}(\mathbf{X}) \leftarrow \mathbf{V}_{:, \leq k}^\top)$$

based on previous theorem, where $\mathbf{X} = \mathbf{U} \Sigma \mathbf{V}^\top$
and k is the hidden layer size

Let's show $\mathbf{h}(\mathbf{X})$ is a linear encoder:

$$\begin{aligned} \mathbf{h}(\mathbf{X}) &= \mathbf{V}_{:, \leq k}^\top \\ &= \mathbf{V}_{:, \leq k}^\top (\mathbf{X}^\top \mathbf{X})^{-1} (\mathbf{X}^\top \mathbf{X}) \end{aligned} \quad \leftarrow \text{multiplying by identity}$$

$$\arg \min_{\mathbf{W}^*, \mathbf{h}(\mathbf{X})} \frac{1}{2} \|\mathbf{X} - \mathbf{W}^* \mathbf{h}(\mathbf{X})\|_F^2 = (\mathbf{W}^* \leftarrow \mathbf{U}_{:, \leq k} \Sigma_{\leq k, \leq k}, \mathbf{h}(\mathbf{X}) \leftarrow \mathbf{V}_{:, \leq k}^\top)$$

based on previous theorem, where $\mathbf{X} = \mathbf{U} \Sigma \mathbf{V}^\top$
and k is the hidden layer size

Let's show $\mathbf{h}(\mathbf{X})$ is a linear encoder:

$$\begin{aligned} \mathbf{h}(\mathbf{X}) &= \mathbf{V}_{:, \leq k}^\top \\ &= \mathbf{V}_{:, \leq k}^\top (\mathbf{X}^\top \mathbf{X})^{-1} (\mathbf{X}^\top \mathbf{X}) && \leftarrow \text{multiplying by identity} \\ &= \mathbf{V}_{:, \leq k}^\top (\mathbf{V} \Sigma^\top \mathbf{U}^\top \mathbf{U} \Sigma \mathbf{V}^\top)^{-1} (\mathbf{V} \Sigma^\top \mathbf{U}^\top \mathbf{X}) && \leftarrow \text{replace with SVD} \end{aligned}$$

$$\arg \min_{\mathbf{W}^*, \mathbf{h}(\mathbf{X})} \frac{1}{2} \|\mathbf{X} - \mathbf{W}^* \mathbf{h}(\mathbf{X})\|_F^2 = (\mathbf{W}^* \leftarrow \mathbf{U}_{:, \leq k} \Sigma_{\leq k, \leq k}, \mathbf{h}(\mathbf{X}) \leftarrow \mathbf{V}_{:, \leq k}^\top)$$

based on previous theorem, where $\mathbf{X} = \mathbf{U} \Sigma \mathbf{V}^\top$
and k is the hidden layer size

Let's show $\mathbf{h}(\mathbf{X})$ is a linear encoder:

$$\begin{aligned} \mathbf{h}(\mathbf{X}) &= \mathbf{V}_{:, \leq k}^\top \\ &= \mathbf{V}_{:, \leq k}^\top (\mathbf{X}^\top \mathbf{X})^{-1} (\mathbf{X}^\top \mathbf{X}) && \leftarrow \text{multiplying by identity} \\ &= \mathbf{V}_{:, \leq k}^\top (\mathbf{V} \Sigma^\top \mathbf{U}^\top \mathbf{U} \Sigma \mathbf{V}^\top)^{-1} (\mathbf{V} \Sigma^\top \mathbf{U}^\top \mathbf{X}) && \leftarrow \text{replace with SVD} \\ &= \mathbf{V}_{:, \leq k}^\top (\mathbf{V} \Sigma^\top \Sigma \mathbf{V}^\top)^{-1} \mathbf{V} \Sigma^\top \mathbf{U}^\top \mathbf{X} && \leftarrow \mathbf{U}^\top \mathbf{U} = \mathbf{I} \text{ (orthonormal)} \end{aligned}$$

$$\arg \min_{\mathbf{W}^*, \mathbf{h}(\mathbf{X})} \frac{1}{2} \|\mathbf{X} - \mathbf{W}^* \mathbf{h}(\mathbf{X})\|_F^2 = (\mathbf{W}^* \leftarrow \mathbf{U}_{:, \leq k} \Sigma_{\leq k, \leq k}, \mathbf{h}(\mathbf{X}) \leftarrow \mathbf{V}_{:, \leq k}^\top)$$

based on previous theorem, where $\mathbf{X} = \mathbf{U} \Sigma \mathbf{V}^\top$
and k is the hidden layer size

Let's show $\mathbf{h}(\mathbf{X})$ is a linear encoder:

$$\begin{aligned} \mathbf{h}(\mathbf{X}) &= \mathbf{V}_{:, \leq k}^\top \\ &= \mathbf{V}_{:, \leq k}^\top (\mathbf{X}^\top \mathbf{X})^{-1} (\mathbf{X}^\top \mathbf{X}) && \leftarrow \text{multiplying by identity} \\ &= \mathbf{V}_{:, \leq k}^\top (\mathbf{V} \Sigma^\top \mathbf{U}^\top \mathbf{U} \Sigma \mathbf{V}^\top)^{-1} (\mathbf{V} \Sigma^\top \mathbf{U}^\top \mathbf{X}) && \leftarrow \text{replace with SVD} \\ &= \mathbf{V}_{:, \leq k}^\top (\mathbf{V} \Sigma^\top \Sigma \mathbf{V}^\top)^{-1} \mathbf{V} \Sigma^\top \mathbf{U}^\top \mathbf{X} && \leftarrow \mathbf{U}^\top \mathbf{U} = \mathbf{I} \text{ (orthonormal)} \\ &= \mathbf{V}_{:, \leq k}^\top \mathbf{V} (\Sigma^\top \Sigma)^{-1} \mathbf{V}^\top \mathbf{V} \Sigma^\top \mathbf{U}^\top \mathbf{X} && \leftarrow \mathbf{V} (\Sigma^\top \Sigma)^{-1} \mathbf{V}^\top \mathbf{V} \Sigma^\top \Sigma \mathbf{V}^\top = \mathbf{I} \end{aligned}$$

$$\arg \min_{\mathbf{W}^*, \mathbf{h}(\mathbf{X})} \frac{1}{2} \|\mathbf{X} - \mathbf{W}^* \mathbf{h}(\mathbf{X})\|_F^2 = (\mathbf{W}^* \leftarrow \mathbf{U}_{:, \leq k} \Sigma_{\leq k, \leq k}, \mathbf{h}(\mathbf{X}) \leftarrow \mathbf{V}_{:, \leq k}^\top)$$

based on previous theorem, where $\mathbf{X} = \mathbf{U} \Sigma \mathbf{V}^\top$
and k is the hidden layer size

Let's show $\mathbf{h}(\mathbf{X})$ is a linear encoder:

$$\begin{aligned} \mathbf{h}(\mathbf{X}) &= \mathbf{V}_{:, \leq k}^\top \\ &= \mathbf{V}_{:, \leq k}^\top (\mathbf{X}^\top \mathbf{X})^{-1} (\mathbf{X}^\top \mathbf{X}) && \leftarrow \text{multiplying by identity} \\ &= \mathbf{V}_{:, \leq k}^\top (\mathbf{V} \Sigma^\top \mathbf{U}^\top \mathbf{U} \Sigma \mathbf{V}^\top)^{-1} (\mathbf{V} \Sigma^\top \mathbf{U}^\top \mathbf{X}) && \leftarrow \text{replace with SVD} \\ &= \mathbf{V}_{:, \leq k}^\top (\mathbf{V} \Sigma^\top \Sigma \mathbf{V}^\top)^{-1} \mathbf{V} \Sigma^\top \mathbf{U}^\top \mathbf{X} && \leftarrow \mathbf{U}^\top \mathbf{U} = \mathbf{I} \text{ (orthonormal)} \\ &= \mathbf{V}_{:, \leq k}^\top \mathbf{V} (\Sigma^\top \Sigma)^{-1} \mathbf{V}^\top \mathbf{V} \Sigma^\top \mathbf{U}^\top \mathbf{X} && \leftarrow \mathbf{V} (\Sigma^\top \Sigma)^{-1} \mathbf{V}^\top \mathbf{V} \Sigma^\top \Sigma \mathbf{V}^\top = \mathbf{I} \\ &= \mathbf{V}_{:, \leq k}^\top \mathbf{V} (\Sigma^\top \Sigma)^{-1} \Sigma^\top \mathbf{U}^\top \mathbf{X} && \leftarrow \mathbf{V}^\top \mathbf{V} = \mathbf{I} \text{ (orthonormal)} \end{aligned}$$

$$\arg \min_{\mathbf{W}^*, \mathbf{h}(\mathbf{X})} \frac{1}{2} \|\mathbf{X} - \mathbf{W}^* \mathbf{h}(\mathbf{X})\|_F^2 = (\mathbf{W}^* \leftarrow \mathbf{U}_{:, \leq k} \Sigma_{\leq k, \leq k}, \mathbf{h}(\mathbf{X}) \leftarrow \mathbf{V}_{:, \leq k}^\top)$$

based on previous theorem, where $\mathbf{X} = \mathbf{U} \Sigma \mathbf{V}^\top$
and k is the hidden layer size

Let's show $\mathbf{h}(\mathbf{X})$ is a linear encoder:

$$\begin{aligned} \mathbf{h}(\mathbf{X}) &= \mathbf{V}_{:, \leq k}^\top \\ &= \mathbf{V}_{:, \leq k}^\top (\mathbf{X}^\top \mathbf{X})^{-1} (\mathbf{X}^\top \mathbf{X}) && \leftarrow \text{multiplying by identity} \\ &= \mathbf{V}_{:, \leq k}^\top (\mathbf{V} \Sigma^\top \mathbf{U}^\top \mathbf{U} \Sigma \mathbf{V}^\top)^{-1} (\mathbf{V} \Sigma^\top \mathbf{U}^\top \mathbf{X}) && \leftarrow \text{replace with SVD} \\ &= \mathbf{V}_{:, \leq k}^\top (\mathbf{V} \Sigma^\top \Sigma \mathbf{V}^\top)^{-1} \mathbf{V} \Sigma^\top \mathbf{U}^\top \mathbf{X} && \leftarrow \mathbf{U}^\top \mathbf{U} = \mathbf{I} \text{ (orthonormal)} \\ &= \mathbf{V}_{:, \leq k}^\top \mathbf{V} (\Sigma^\top \Sigma)^{-1} \mathbf{V}^\top \mathbf{V} \Sigma^\top \mathbf{U}^\top \mathbf{X} && \leftarrow \mathbf{V} (\Sigma^\top \Sigma)^{-1} \mathbf{V}^\top \mathbf{V} \Sigma^\top \Sigma \mathbf{V}^\top = \mathbf{I} \\ &= \mathbf{V}_{:, \leq k}^\top \mathbf{V} (\Sigma^\top \Sigma)^{-1} \Sigma^\top \mathbf{U}^\top \mathbf{X} && \leftarrow \mathbf{V}^\top \mathbf{V} = \mathbf{I} \text{ (orthonormal)} \\ &= \mathbf{I}_{\leq k, \cdot} (\Sigma^\top \Sigma)^{-1} \Sigma^\top \mathbf{U}^\top \mathbf{X} && \leftarrow \text{idem} \end{aligned}$$

$$\arg \min_{\mathbf{W}^*, \mathbf{h}(\mathbf{X})} \frac{1}{2} \|\mathbf{X} - \mathbf{W}^* \mathbf{h}(\mathbf{X})\|_F^2 = (\mathbf{W}^* \leftarrow \mathbf{U}_{:, \leq k} \Sigma_{\leq k, \leq k}, \mathbf{h}(\mathbf{X}) \leftarrow \mathbf{V}_{:, \leq k}^\top)$$

based on previous theorem, where $\mathbf{X} = \mathbf{U} \Sigma \mathbf{V}^\top$
and k is the hidden layer size

Let's show $\mathbf{h}(\mathbf{X})$ is a linear encoder:

$$\begin{aligned} \mathbf{h}(\mathbf{X}) &= \mathbf{V}_{:, \leq k}^\top \\ &= \mathbf{V}_{:, \leq k}^\top (\mathbf{X}^\top \mathbf{X})^{-1} (\mathbf{X}^\top \mathbf{X}) && \leftarrow \text{multiplying by identity} \\ &= \mathbf{V}_{:, \leq k}^\top (\mathbf{V} \Sigma^\top \mathbf{U}^\top \mathbf{U} \Sigma \mathbf{V}^\top)^{-1} (\mathbf{V} \Sigma^\top \mathbf{U}^\top \mathbf{X}) && \leftarrow \text{replace with SVD} \\ &= \mathbf{V}_{:, \leq k}^\top (\mathbf{V} \Sigma^\top \Sigma \mathbf{V}^\top)^{-1} \mathbf{V} \Sigma^\top \mathbf{U}^\top \mathbf{X} && \leftarrow \mathbf{U}^\top \mathbf{U} = \mathbf{I} \text{ (orthonormal)} \\ &= \mathbf{V}_{:, \leq k}^\top \mathbf{V} (\Sigma^\top \Sigma)^{-1} \mathbf{V}^\top \mathbf{V} \Sigma^\top \mathbf{U}^\top \mathbf{X} && \leftarrow \mathbf{V} (\Sigma^\top \Sigma)^{-1} \mathbf{V}^\top \mathbf{V} \Sigma^\top \Sigma \mathbf{V}^\top = \mathbf{I} \\ &= \mathbf{V}_{:, \leq k}^\top \mathbf{V} (\Sigma^\top \Sigma)^{-1} \Sigma^\top \mathbf{U}^\top \mathbf{X} && \leftarrow \mathbf{V}^\top \mathbf{V} = \mathbf{I} \text{ (orthonormal)} \\ &= \mathbf{I}_{\leq k, \cdot} (\Sigma^\top \Sigma)^{-1} \Sigma^\top \mathbf{U}^\top \mathbf{X} && \leftarrow \text{idem} \\ &= \mathbf{I}_{\leq k, \cdot} \Sigma^{-1} (\Sigma^\top)^{-1} \Sigma^\top \mathbf{U}^\top \mathbf{X} && \leftarrow (\Sigma^\top \Sigma)^{-1} = \Sigma^{-1} (\Sigma^\top)^{-1} \end{aligned}$$

$$\arg \min_{\mathbf{W}^*, \mathbf{h}(\mathbf{X})} \frac{1}{2} \|\mathbf{X} - \mathbf{W}^* \mathbf{h}(\mathbf{X})\|_F^2 = (\mathbf{W}^* \leftarrow \mathbf{U}_{:, \leq k} \Sigma_{\leq k, \leq k}, \mathbf{h}(\mathbf{X}) \leftarrow \mathbf{V}_{:, \leq k}^\top)$$

based on previous theorem, where $\mathbf{X} = \mathbf{U} \Sigma \mathbf{V}^\top$
and k is the hidden layer size

Let's show $\mathbf{h}(\mathbf{X})$ is a linear encoder:

$$\begin{aligned} \mathbf{h}(\mathbf{X}) &= \mathbf{V}_{:, \leq k}^\top \\ &= \mathbf{V}_{:, \leq k}^\top (\mathbf{X}^\top \mathbf{X})^{-1} (\mathbf{X}^\top \mathbf{X}) && \leftarrow \text{multiplying by identity} \\ &= \mathbf{V}_{:, \leq k}^\top (\mathbf{V} \Sigma^\top \mathbf{U}^\top \mathbf{U} \Sigma \mathbf{V}^\top)^{-1} (\mathbf{V} \Sigma^\top \mathbf{U}^\top \mathbf{X}) && \leftarrow \text{replace with SVD} \\ &= \mathbf{V}_{:, \leq k}^\top (\mathbf{V} \Sigma^\top \Sigma \mathbf{V}^\top)^{-1} \mathbf{V} \Sigma^\top \mathbf{U}^\top \mathbf{X} && \leftarrow \mathbf{U}^\top \mathbf{U} = \mathbf{I} \text{ (orthonormal)} \\ &= \mathbf{V}_{:, \leq k}^\top \mathbf{V} (\Sigma^\top \Sigma)^{-1} \mathbf{V}^\top \mathbf{V} \Sigma^\top \mathbf{U}^\top \mathbf{X} && \leftarrow \mathbf{V} (\Sigma^\top \Sigma)^{-1} \mathbf{V}^\top \mathbf{V} \Sigma^\top \Sigma \mathbf{V}^\top = \mathbf{I} \\ &= \mathbf{V}_{:, \leq k}^\top \mathbf{V} (\Sigma^\top \Sigma)^{-1} \Sigma^\top \mathbf{U}^\top \mathbf{X} && \leftarrow \mathbf{V}^\top \mathbf{V} = \mathbf{I} \text{ (orthonormal)} \\ &= \mathbf{I}_{\leq k, \cdot} (\Sigma^\top \Sigma)^{-1} \Sigma^\top \mathbf{U}^\top \mathbf{X} && \leftarrow \text{idem} \\ &= \mathbf{I}_{\leq k, \cdot} \Sigma^{-1} (\Sigma^\top)^{-1} \Sigma^\top \mathbf{U}^\top \mathbf{X} && \leftarrow (\Sigma^\top \Sigma)^{-1} = \Sigma^{-1} (\Sigma^\top)^{-1} \\ &= \mathbf{I}_{\leq k, \cdot} \Sigma^{-1} \mathbf{U}^\top \mathbf{X} \end{aligned}$$

$$\arg \min_{\mathbf{W}^*, \mathbf{h}(\mathbf{X})} \frac{1}{2} \|\mathbf{X} - \mathbf{W}^* \mathbf{h}(\mathbf{X})\|_F^2 = (\mathbf{W}^* \leftarrow \mathbf{U}_{:, \leq k} \Sigma_{\leq k, \leq k}, \mathbf{h}(\mathbf{X}) \leftarrow \mathbf{V}_{:, \leq k}^\top)$$

based on previous theorem, where $\mathbf{X} = \mathbf{U} \Sigma \mathbf{V}^\top$
and k is the hidden layer size

Let's show $\mathbf{h}(\mathbf{X})$ is a linear encoder:

$$\begin{aligned} \mathbf{h}(\mathbf{X}) &= \mathbf{V}_{:, \leq k}^\top \\ &= \mathbf{V}_{:, \leq k}^\top (\mathbf{X}^\top \mathbf{X})^{-1} (\mathbf{X}^\top \mathbf{X}) && \leftarrow \text{multiplying by identity} \\ &= \mathbf{V}_{:, \leq k}^\top (\mathbf{V} \Sigma^\top \mathbf{U}^\top \mathbf{U} \Sigma \mathbf{V}^\top)^{-1} (\mathbf{V} \Sigma^\top \mathbf{U}^\top \mathbf{X}) && \leftarrow \text{replace with SVD} \\ &= \mathbf{V}_{:, \leq k}^\top (\mathbf{V} \Sigma^\top \Sigma \mathbf{V}^\top)^{-1} \mathbf{V} \Sigma^\top \mathbf{U}^\top \mathbf{X} && \leftarrow \mathbf{U}^\top \mathbf{U} = \mathbf{I} \text{ (orthonormal)} \\ &= \mathbf{V}_{:, \leq k}^\top \mathbf{V} (\Sigma^\top \Sigma)^{-1} \mathbf{V}^\top \mathbf{V} \Sigma^\top \mathbf{U}^\top \mathbf{X} && \leftarrow \mathbf{V} (\Sigma^\top \Sigma)^{-1} \mathbf{V}^\top \mathbf{V} \Sigma^\top \Sigma \mathbf{V}^\top = \mathbf{I} \\ &= \mathbf{V}_{:, \leq k}^\top \mathbf{V} (\Sigma^\top \Sigma)^{-1} \Sigma^\top \mathbf{U}^\top \mathbf{X} && \leftarrow \mathbf{V}^\top \mathbf{V} = \mathbf{I} \text{ (orthonormal)} \\ &= \mathbf{I}_{\leq k, \cdot} (\Sigma^\top \Sigma)^{-1} \Sigma^\top \mathbf{U}^\top \mathbf{X} && \leftarrow \text{idem} \\ &= \mathbf{I}_{\leq k, \cdot} \Sigma^{-1} (\Sigma^\top)^{-1} \Sigma^\top \mathbf{U}^\top \mathbf{X} && \leftarrow (\Sigma^\top \Sigma)^{-1} = \Sigma^{-1} (\Sigma^\top)^{-1} \\ &= \mathbf{I}_{\leq k, \cdot} \Sigma^{-1} \mathbf{U}^\top \mathbf{X} \\ &= \Sigma_{\leq k, \leq k}^{-1} (\mathbf{U}_{:, \leq k})^\top \mathbf{X} && \leftarrow \text{multiplying by } \mathbf{I}_{\leq k, \cdot} \text{ selects the } k \text{ first rows} \end{aligned}$$

$$\arg \min_{\mathbf{W}^*, \mathbf{h}(\mathbf{X})} \frac{1}{2} \|\mathbf{X} - \mathbf{W}^* \mathbf{h}(\mathbf{X})\|_F^2 = (\mathbf{W}^* \leftarrow \mathbf{U}_{:, \leq k} \Sigma_{\leq k, \leq k}, \mathbf{h}(\mathbf{X}) \leftarrow \mathbf{V}_{:, \leq k}^\top)$$

based on previous theorem, where $\mathbf{X} = \mathbf{U} \Sigma \mathbf{V}^\top$
and k is the hidden layer size

Let's show $\mathbf{h}(\mathbf{X})$ is a linear encoder:

$$\begin{aligned} \mathbf{h}(\mathbf{X}) &= \mathbf{V}_{:, \leq k}^\top \\ &= \mathbf{V}_{:, \leq k}^\top (\mathbf{X}^\top \mathbf{X})^{-1} (\mathbf{X}^\top \mathbf{X}) && \leftarrow \text{multiplying by identity} \\ &= \mathbf{V}_{:, \leq k}^\top (\mathbf{V} \Sigma^\top \mathbf{U}^\top \mathbf{U} \Sigma \mathbf{V}^\top)^{-1} (\mathbf{V} \Sigma^\top \mathbf{U}^\top \mathbf{X}) && \leftarrow \text{replace with SVD} \\ &= \mathbf{V}_{:, \leq k}^\top (\mathbf{V} \Sigma^\top \Sigma \mathbf{V}^\top)^{-1} \mathbf{V} \Sigma^\top \mathbf{U}^\top \mathbf{X} && \leftarrow \mathbf{U}^\top \mathbf{U} = \mathbf{I} \text{ (orthonormal)} \\ &= \mathbf{V}_{:, \leq k}^\top \mathbf{V} (\Sigma^\top \Sigma)^{-1} \mathbf{V}^\top \mathbf{V} \Sigma^\top \mathbf{U}^\top \mathbf{X} && \leftarrow \mathbf{V} (\Sigma^\top \Sigma)^{-1} \mathbf{V}^\top \mathbf{V} \Sigma^\top \Sigma \mathbf{V}^\top = \mathbf{I} \\ &= \mathbf{V}_{:, \leq k}^\top \mathbf{V} (\Sigma^\top \Sigma)^{-1} \Sigma^\top \mathbf{U}^\top \mathbf{X} && \leftarrow \mathbf{V}^\top \mathbf{V} = \mathbf{I} \text{ (orthonormal)} \\ &= \mathbf{I}_{\leq k, \cdot} (\Sigma^\top \Sigma)^{-1} \Sigma^\top \mathbf{U}^\top \mathbf{X} && \leftarrow \text{idem} \\ &= \mathbf{I}_{\leq k, \cdot} \Sigma^{-1} (\Sigma^\top)^{-1} \Sigma^\top \mathbf{U}^\top \mathbf{X} && \leftarrow (\Sigma^\top \Sigma)^{-1} = \Sigma^{-1} (\Sigma^\top)^{-1} \\ &= \mathbf{I}_{\leq k, \cdot} \Sigma^{-1} \mathbf{U}^\top \mathbf{X} \\ &= \underbrace{\Sigma_{\leq k, \leq k}^{-1} (\mathbf{U}_{:, \leq k})^\top}_{\text{this is a linear encoder}} \mathbf{X} && \leftarrow \text{multiplying by } \mathbf{I}_{\leq k, \cdot} \text{ selects the } k \text{ first rows} \end{aligned}$$

AUTOENCODER

Topics: optimality of a linear autoencoder

- So an optimal pair of encoder and decoder is

$$\mathbf{h}(\mathbf{x}) = \underbrace{\left(\Sigma_{\leq k, \leq k}^{-1} (\mathbf{U}_{\cdot, \leq k})^\top \right)}_{\mathbf{W}} \mathbf{x} \quad \hat{\mathbf{x}} = \underbrace{(\mathbf{U}_{\cdot, \leq k} \Sigma_{\leq k, \leq k})}_{\mathbf{W}^*} \mathbf{h}(\mathbf{x})$$

AUTOENCODER

Topics: optimality of a linear autoencoder

- So an optimal pair of encoder and decoder is

$$\mathbf{h}(\mathbf{x}) = \underbrace{\left(\Sigma_{\leq k, \leq k}^{-1} (\mathbf{U}_{\cdot, \leq k})^\top \right)}_{\mathbf{W}} \mathbf{x} \quad \hat{\mathbf{x}} = \underbrace{(\mathbf{U}_{\cdot, \leq k} \Sigma_{\leq k, \leq k})}_{\mathbf{W}^*} \mathbf{h}(\mathbf{x})$$

- for the sum of squared difference error
- for an autoencoder with a linear decoder
- where optimality means “has the lowest training reconstruction error”

AUTOENCODER

Topics: optimality of a linear autoencoder

- So an optimal pair of encoder and decoder is

$$\mathbf{h}(\mathbf{x}) = \underbrace{\left(\Sigma_{\leq k, \leq k}^{-1} (\mathbf{U}_{\cdot, \leq k})^\top \right)}_{\mathbf{W}} \mathbf{x} \quad \hat{\mathbf{x}} = \underbrace{(\mathbf{U}_{\cdot, \leq k} \Sigma_{\leq k, \leq k})}_{\mathbf{W}^*} \mathbf{h}(\mathbf{x})$$

- for the sum of squared difference error
- for an autoencoder with a linear decoder
- where optimality means “has the lowest training reconstruction error”
- If inputs are normalized as follows: $\mathbf{x}^{(t)} \leftarrow \frac{1}{\sqrt{T}} \left(\mathbf{x}^{(t)} - \frac{1}{T} \sum_{t'=1}^T \mathbf{x}^{(t')} \right)$
 - encoder corresponds to Principal Component Analysis (PCA)
 - singular values and (left) vectors = the eigenvalues/vectors of covariance matrix

Note: In fact, if we were to construct a linear network (ie. without the use of nonlinear activation functions at each layer) we would observe a similar dimensionality reduction as observed in PCA. See Geoffrey Hinton's discussion of this here.

<https://www.jeremyjordan.me/autoencoders/>

Note: In fact, if we were to construct a linear network (ie. without the use of nonlinear activation functions at each layer) we would observe a similar dimensionality reduction as observed in PCA. See Geoffrey Hinton's discussion of this here.

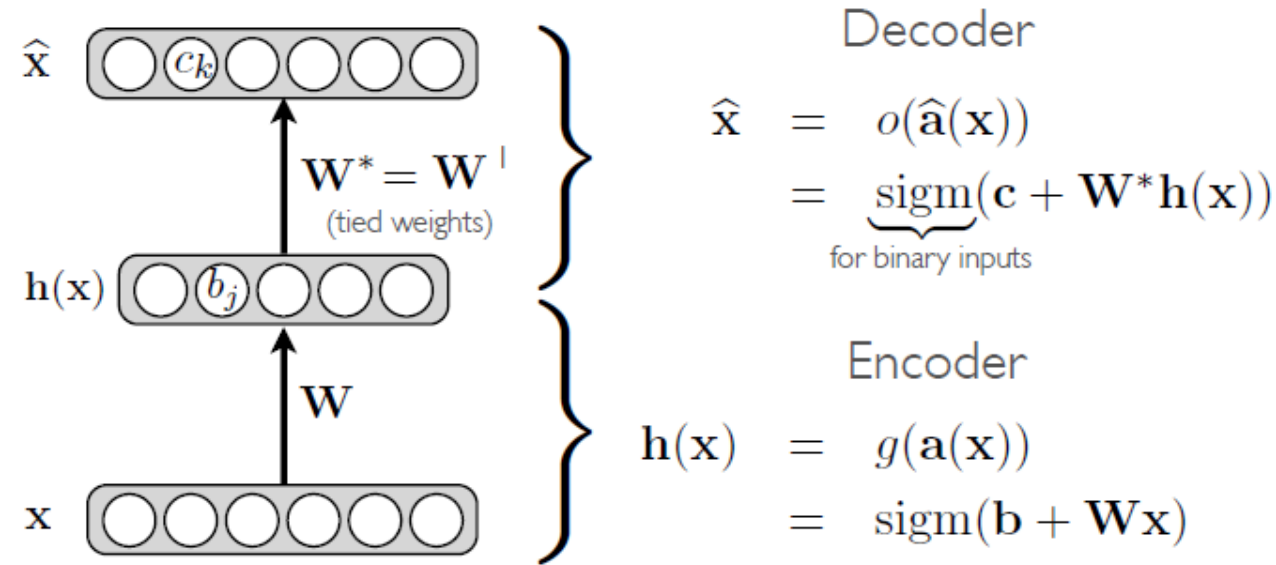
The ideal autoencoder model balances the following:

- Sensitive to the inputs enough to accurately build a reconstruction.
- Insensitive enough to the inputs that the model doesn't simply memorize or overfit the training data.

<https://www.jeremyjordan.me/autoencoders/>

Number of Hidden Units

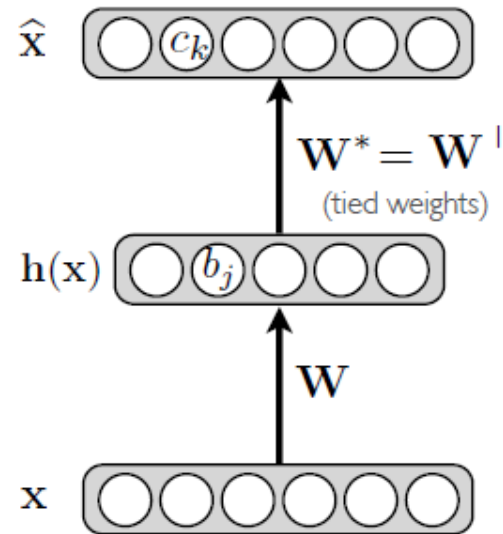
- Undercomplete versus Overcomplete



UNDERCOMPLETE HIDDEN LAYER

Topics: undercomplete representation

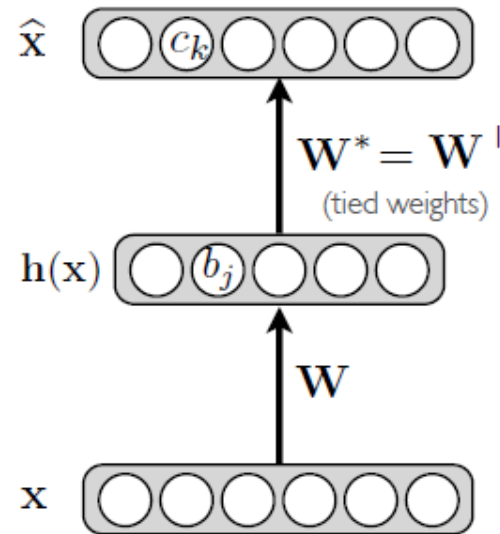
- Hidden layer is undercomplete if smaller than the input layer
 - hidden layer “compresses” the input
 - will compress well only for the training distribution



UNDERCOMPLETE HIDDEN LAYER

Topics: undercomplete representation

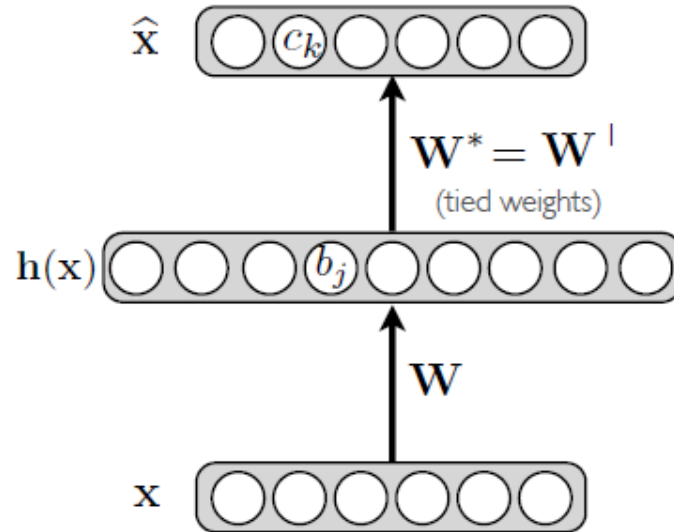
- Hidden layer is undercomplete if smaller than the input layer
 - hidden layer “compresses” the input
 - will compress well only for the training distribution
- Hidden units will be
 - good features for the training distribution → 
 - but bad for other types of input → 



OVERCOMPLETE HIDDEN LAYER

Topics: overcomplete representation

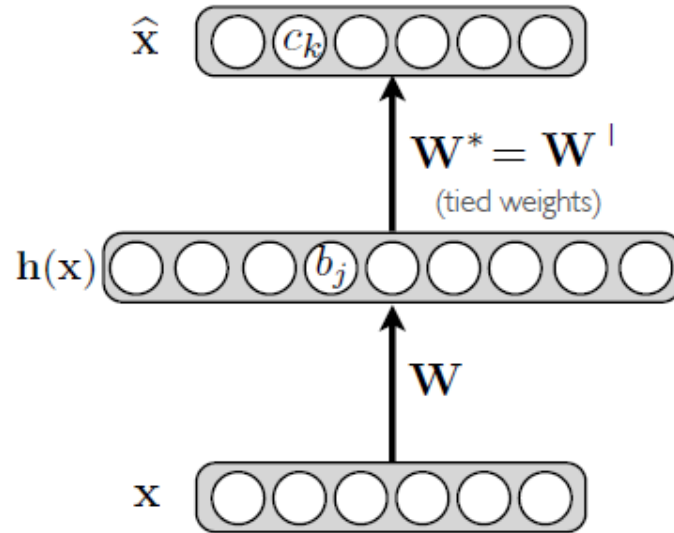
- Hidden layer is overcomplete if greater than the input layer
 - no compression in hidden layer
 - each hidden unit could copy a different input component



OVERCOMPLETE HIDDEN LAYER

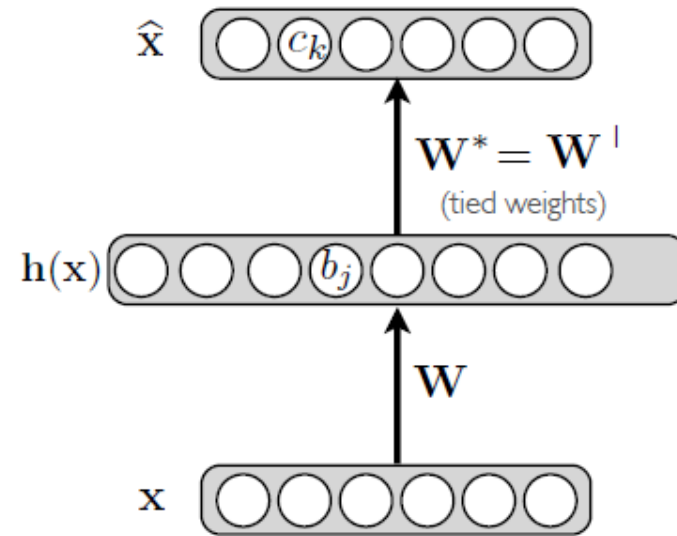
Topics: overcomplete representation

- Hidden layer is overcomplete if greater than the input layer
 - no compression in hidden layer
 - each hidden unit could copy a different input component
- No guarantee that the hidden units will extract meaningful structure



DENOISING AUTOENCODER

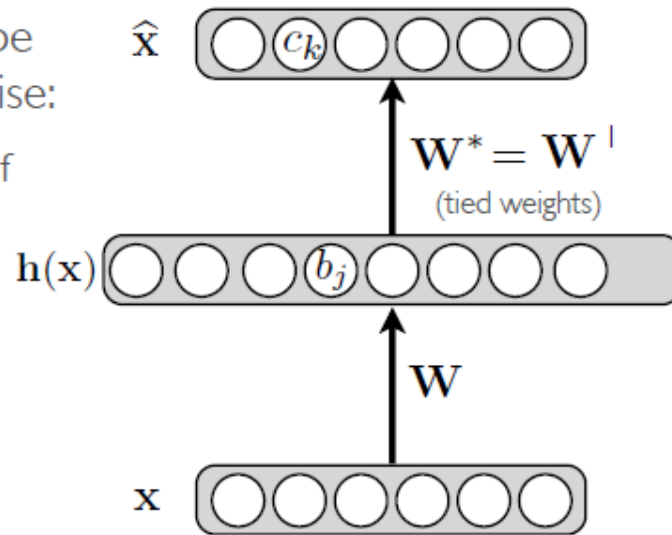
Topics: denoising autoencoder



DENOISING AUTOENCODER

Topics: denoising autoencoder

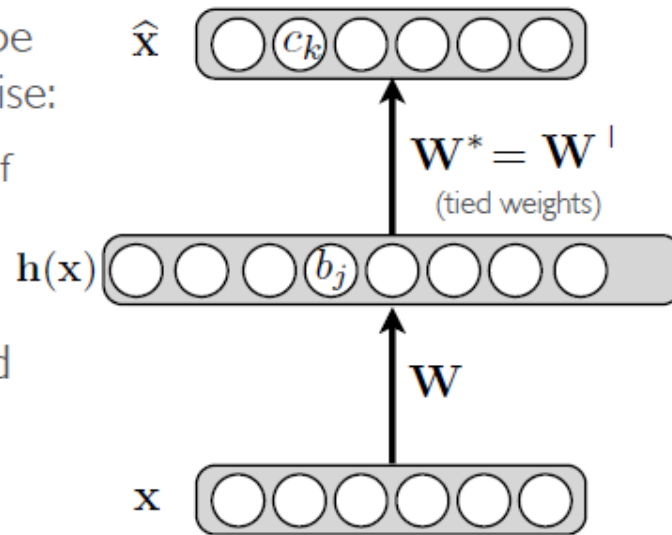
- Idea: representation should be robust to introduction of noise:
 - random assignment of subset of inputs to 0, with probability ν
 - Gaussian additive noise



DENOISING AUTOENCODER

Topics: denoising autoencoder

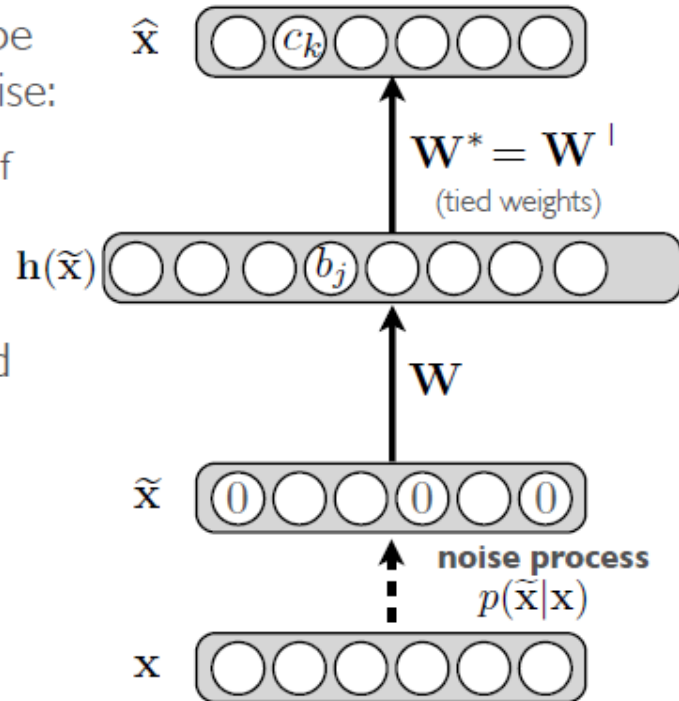
- Idea: representation should be robust to introduction of noise:
 - random assignment of subset of inputs to 0, with probability ν
 - Gaussian additive noise
- Reconstruction $\hat{\mathbf{x}}$ computed from the corrupted input $\tilde{\mathbf{x}}$
- Loss function compares $\hat{\mathbf{x}}$ reconstruction with the **noiseless input $\mathbf{x}$**



DENOISING AUTOENCODER

Topics: denoising autoencoder

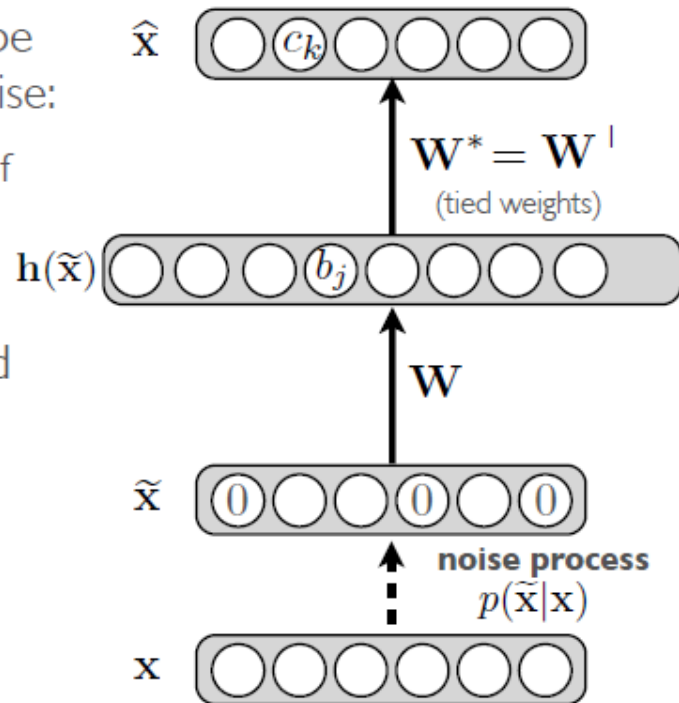
- Idea: representation should be robust to introduction of noise:
 - random assignment of subset of inputs to 0, with probability ν
 - Gaussian additive noise
- Reconstruction $\hat{\mathbf{x}}$ computed from the corrupted input $\tilde{\mathbf{x}}$
- Loss function compares $\hat{\mathbf{x}}$ reconstruction with the **noiseless input $\mathbf{x}$**

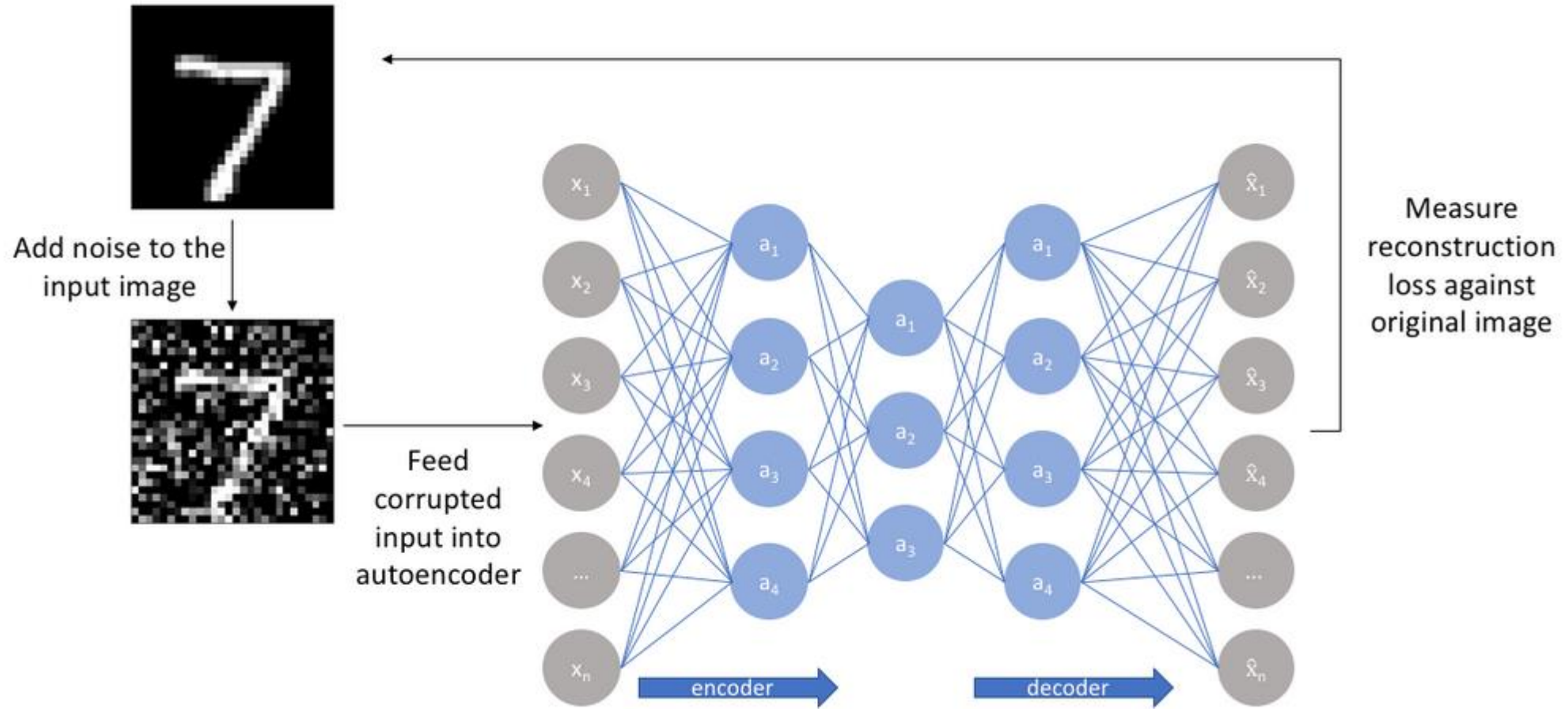


DENOISING AUTOENCODER

Topics: denoising autoencoder

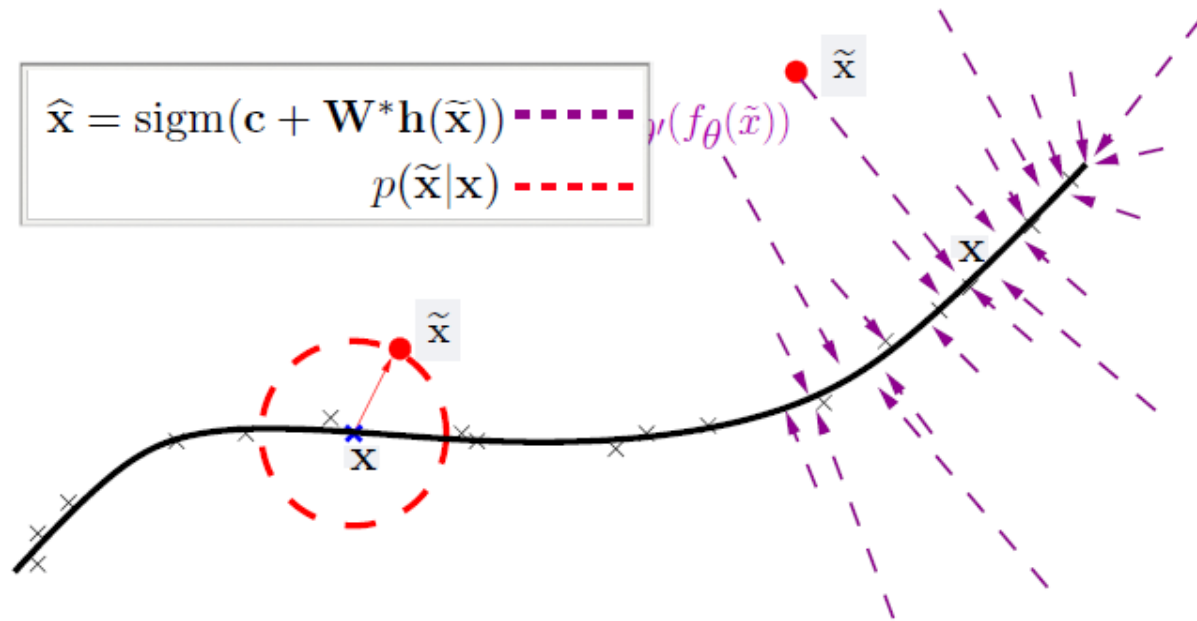
- Idea: representation should be robust to introduction of noise:
 - random assignment of subset of inputs to 0, with probability ν
 - Gaussian additive noise
- Reconstruction $\hat{\mathbf{x}}$ computed from the corrupted input $\tilde{\mathbf{x}}$
- Loss function compares $\hat{\mathbf{x}}$ reconstruction with the **noiseless input $\mathbf{x}$**



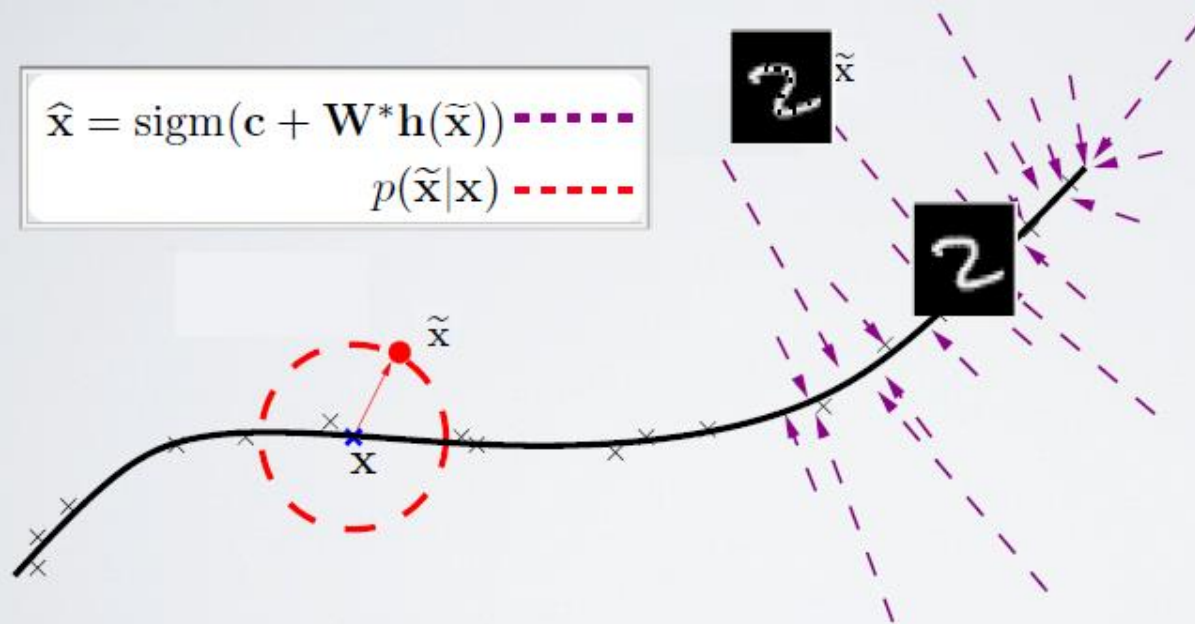


<https://www.jeremyjordan.me/autoencoders/>

DENOISING AUTOENCODER



DENOISING AUTOENCODER

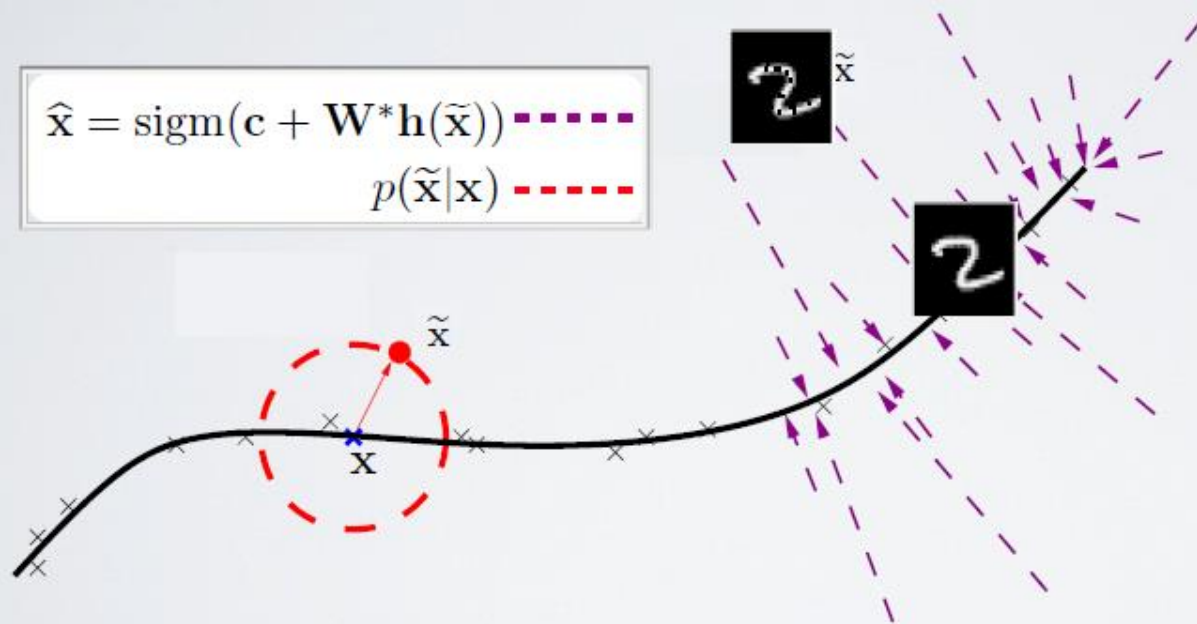


DENOISING AUTOENCODER

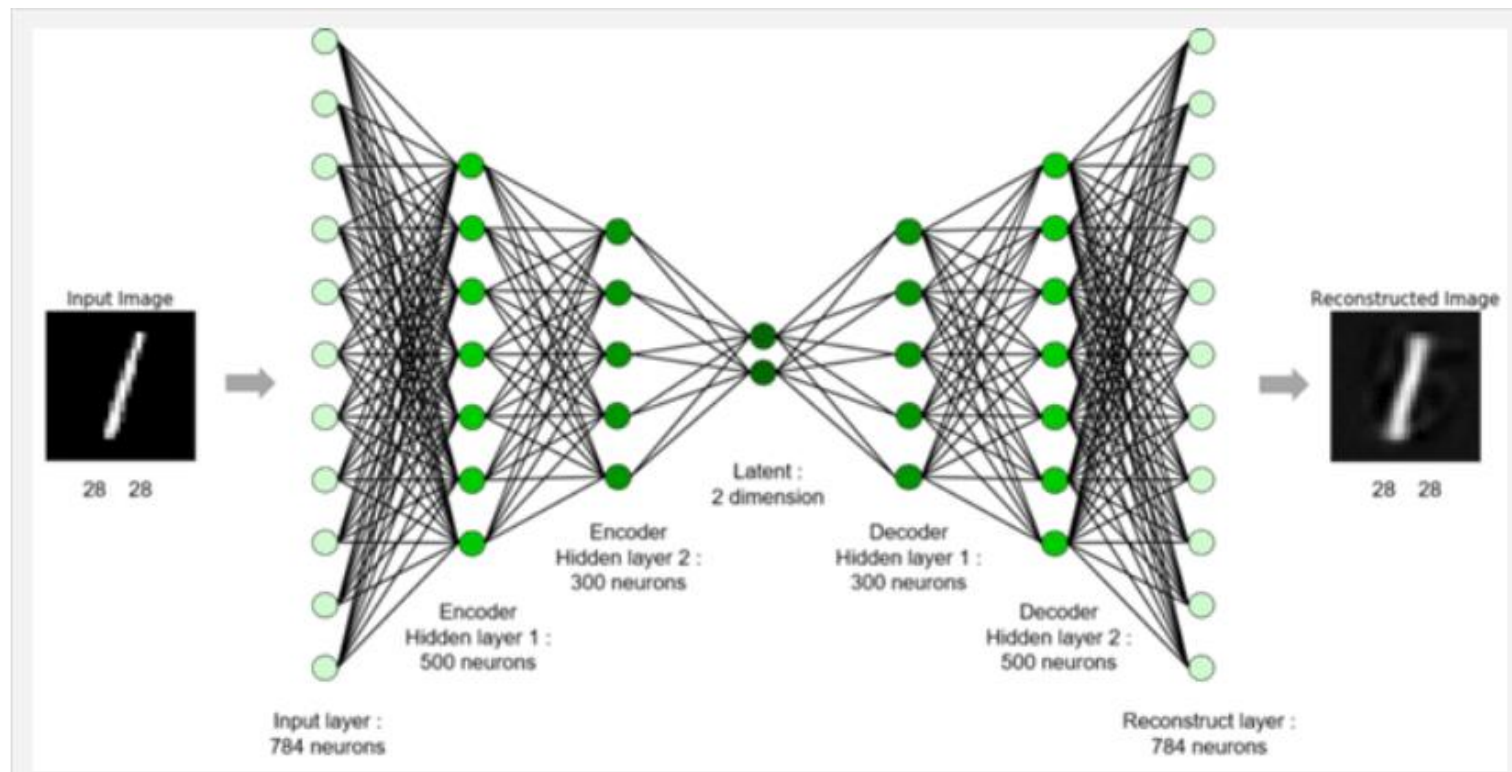
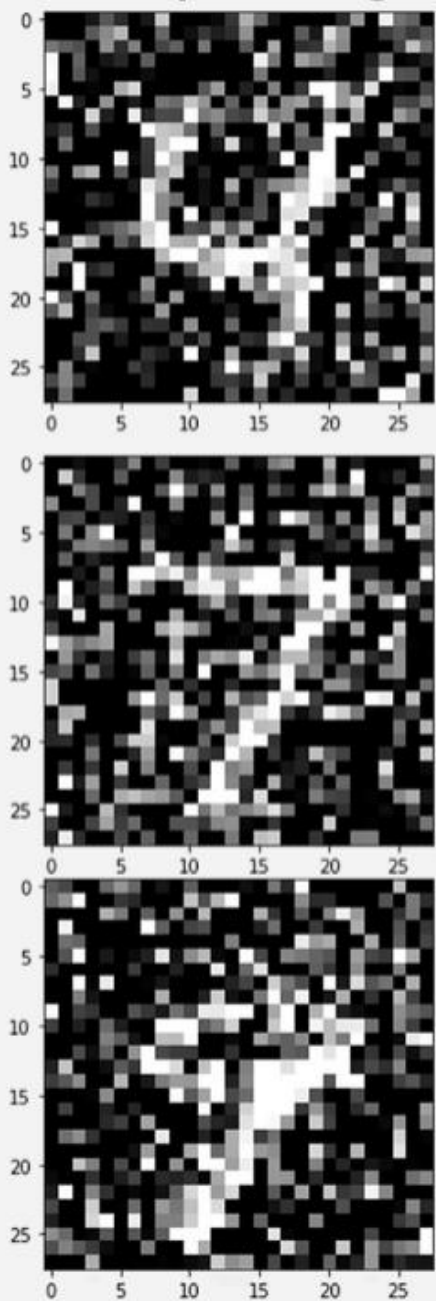


$$\hat{x} = \text{sigm}(\mathbf{c} + \mathbf{W}^* \mathbf{h}(\tilde{x}))$$

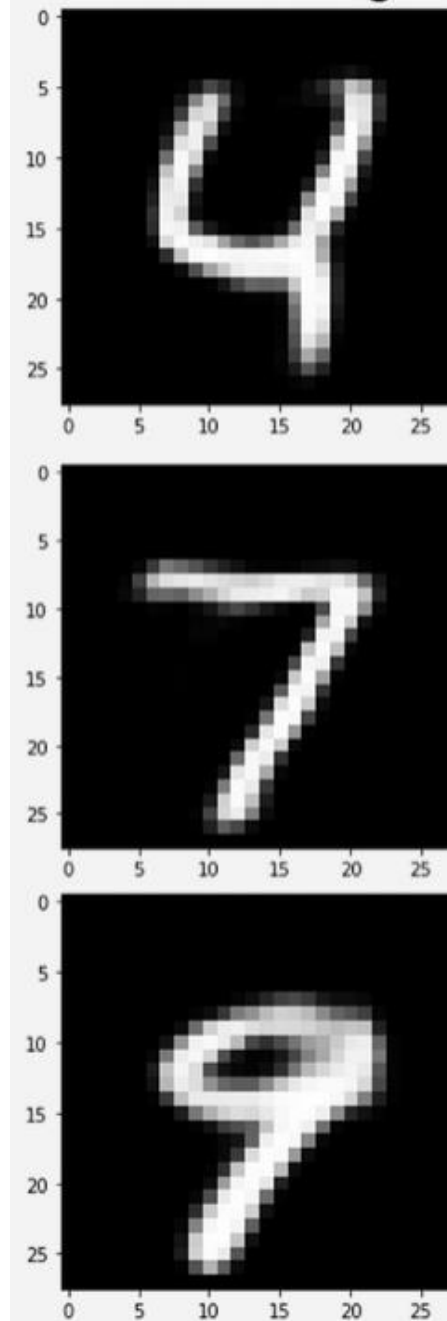
$$p(\tilde{x}|\mathbf{x})$$



Corrupted image



Predicted image



Extracting and Composing Robust Features with Denoising Autoencoders

Pascal Vincent

Hugo Larochelle

Yoshua Bengio

Pierre-Antoine Manzagol

VINCENTP@IRO.UMONTREAL.CA

LAROCHEH@IRO.UMONTREAL.CA

BENGIOY@IRO.UMONTREAL.CA

MANZAGOP@IRO.UMONTREAL.CA

Université de Montréal, Dept. IRO, CP 6128, Succ. Centre-Ville, Montréal, Québec, H3C 3J7, Canada

<https://dl.acm.org/doi/pdf/10.1145/1390156.1390294>

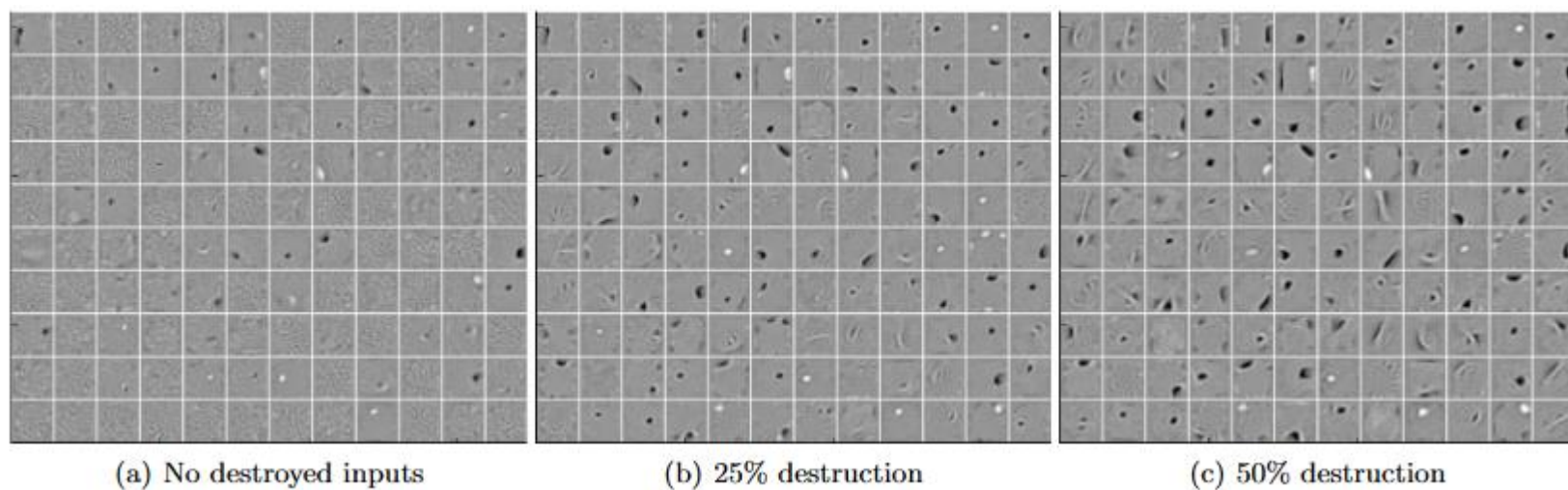
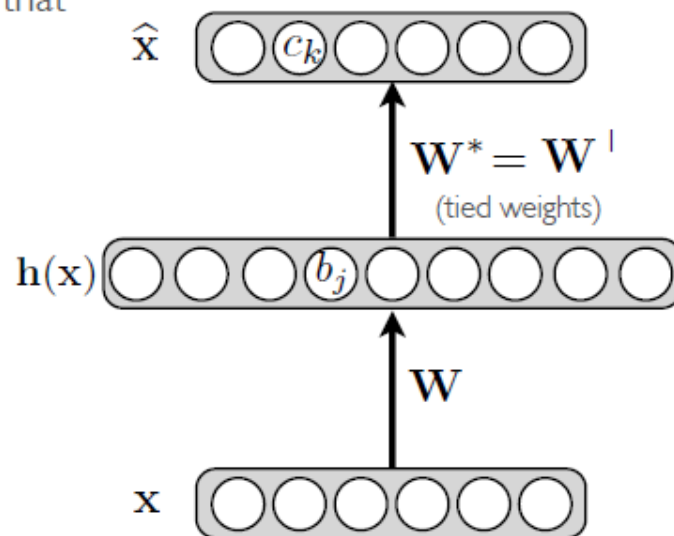


Figure 3. Filters obtained after training the first denoising autoencoder.
(a-c) show some of the filters obtained after training a denoising autoencoder on MNIST samples, with increasing destruction levels ν . The filters at the same position in the three images are related only by the fact that the autoencoders were started from the same random initialization point.

CONTRACTIVE AUTOENCODER

Topics: contractive autoencoder

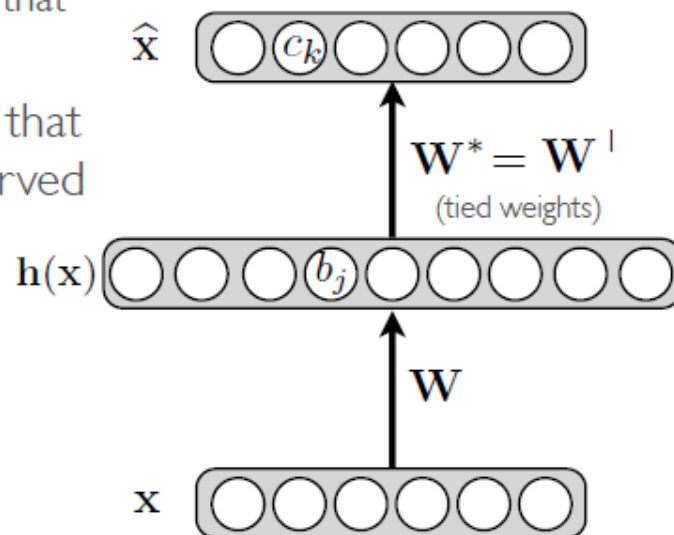
- Alternative approach to avoid uninteresting solutions
 - add an explicit term in the loss that penalizes that solution



CONTRACTIVE AUTOENCODER

Topics: contractive autoencoder

- Alternative approach to avoid uninteresting solutions
 - add an explicit term in the loss that penalizes that solution
- We wish to extract features that **only** reflect variations observed in the training set
 - we'd like to be invariant to the other variations



Because we're explicitly encouraging our model to learn an encoding in which similar inputs have similar encodings, we're essentially forcing the model to learn how to **contract** a neighborhood of inputs into a smaller neighborhood of outputs. Notice how the slope (ie. derivative) of the reconstructed data is essentially zero for local neighborhoods of input data.

We can accomplish this by constructing a loss term which penalizes large *derivatives* of our *hidden layer activations* with respect to the input training examples, essentially penalizing instances where a small change in the input leads to a large change in the encoding space.

For m observations and n hidden layer nodes, we can calculate these values as follows.

$$\|A\|_F = \sqrt{\sum_{i=1}^m \sum_{j=1}^n |a_{ij}|^2}$$

$$\mathbf{J} = \begin{bmatrix} \frac{\delta a_1^{(h)}(x)}{\delta x_1} & \dots & \frac{\delta a_1^{(h)}(x)}{\delta x_m} \\ \vdots & \ddots & \vdots \\ \frac{\delta a_n^{(h)}(x)}{\delta x_1} & \dots & \frac{\delta a_n^{(h)}(x)}{\delta x_m} \end{bmatrix}$$

Written more succinctly, we can define our complete loss function as

$$\mathcal{L}(x, \hat{x}) + \lambda \sum_i \left\| \nabla_x a_i^{(h)}(x) \right\|^2$$

CONTRACTIVE AUTOENCODER

Topics: contractive autoencoder

- New loss function:

$$\underbrace{l(f(\mathbf{x}^{(t)}))}_{\text{autoencoder reconstruction}} + \lambda \underbrace{\|\nabla_{\mathbf{x}^{(t)}} \mathbf{h}(\mathbf{x}^{(t)})\|_F^2}_{\text{Jacobian of encoder}}$$

CONTRACTIVE AUTOENCODER

Topics: contractive autoencoder

- New loss function:

$$\underbrace{l(f(\mathbf{x}^{(t)}))}_{\text{autoencoder reconstruction}} + \lambda \underbrace{\|\nabla_{\mathbf{x}^{(t)}} \mathbf{h}(\mathbf{x}^{(t)})\|_F^2}_{\text{Jacobian of encoder}}$$

- where, for binary observations:

$$l(f(\mathbf{x}^{(t)})) = -\sum_k \left(x_k^{(t)} \log(\hat{x}_k^{(t)}) + (1 - x_k^{(t)}) \log(1 - \hat{x}_k^{(t)}) \right)$$

$$\|\nabla_{\mathbf{x}^{(t)}} \mathbf{h}(\mathbf{x}^{(t)})\|_F^2 = \sum_j \sum_k \left(\frac{\partial h(\mathbf{x}^{(t)})_j}{\partial x_k^{(t)}} \right)^2$$

CONTRACTIVE AUTOENCODER

Topics: contractive autoencoder

- New loss function:

$$\underbrace{l(f(\mathbf{x}^{(t)}))}_{\text{autoencoder reconstruction}} + \lambda \underbrace{\|\nabla_{\mathbf{x}^{(t)}} \mathbf{h}(\mathbf{x}^{(t)})\|_F^2}_{\text{Jacobian of encoder}}$$

- where, for binary observations:

$$l(f(\mathbf{x}^{(t)})) = -\sum_k \left(x_k^{(t)} \log(\hat{x}_k^{(t)}) + (1 - x_k^{(t)}) \log(1 - \hat{x}_k^{(t)}) \right) \left. \vphantom{\sum_k} \right\} \begin{array}{l} \text{encoder keeps} \\ \text{good information} \end{array}$$

$$\|\nabla_{\mathbf{x}^{(t)}} \mathbf{h}(\mathbf{x}^{(t)})\|_F^2 = \sum_j \sum_k \left(\frac{\partial h(\mathbf{x}^{(t)})_j}{\partial x_k^{(t)}} \right)^2$$

CONTRACTIVE AUTOENCODER

Topics: contractive autoencoder

- New loss function:

$$\underbrace{l(f(\mathbf{x}^{(t)}))}_{\text{autoencoder reconstruction}} + \lambda \underbrace{\|\nabla_{\mathbf{x}^{(t)}} \mathbf{h}(\mathbf{x}^{(t)})\|_F^2}_{\text{Jacobian of encoder}}$$

- where, for binary observations:

$$l(f(\mathbf{x}^{(t)})) = -\sum_k \left(x_k^{(t)} \log(\hat{x}_k^{(t)}) + (1 - x_k^{(t)}) \log(1 - \hat{x}_k^{(t)}) \right) \left. \vphantom{\sum_k} \right\} \begin{array}{l} \text{encoder keeps} \\ \text{good information} \end{array}$$

$$\|\nabla_{\mathbf{x}^{(t)}} \mathbf{h}(\mathbf{x}^{(t)})\|_F^2 = \sum_j \sum_k \left(\frac{\partial h(\mathbf{x}^{(t)})_j}{\partial x_k^{(t)}} \right)^2 \left. \vphantom{\sum_j \sum_k} \right\} \begin{array}{l} \text{encoder throws} \\ \text{away all information} \end{array}$$

CONTRACTIVE AUTOENCODER

Topics: contractive autoencoder

- New loss function:

$$\underbrace{l(f(\mathbf{x}^{(t)}))}_{\text{autoencoder reconstruction}} + \lambda \underbrace{\|\nabla_{\mathbf{x}^{(t)}} \mathbf{h}(\mathbf{x}^{(t)})\|_F^2}_{\text{Jacobian of encoder}}$$

- where, for binary observations:

$$l(f(\mathbf{x}^{(t)})) = -\sum_k \left(x_k^{(t)} \log(\hat{x}_k^{(t)}) + (1 - x_k^{(t)}) \log(1 - \hat{x}_k^{(t)}) \right) \left. \vphantom{\sum_k} \right\} \text{encoder keeps good information}$$

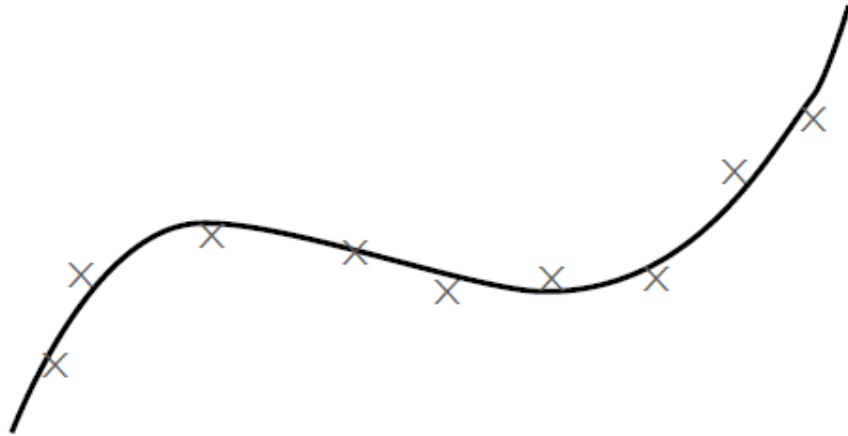
$$\|\nabla_{\mathbf{x}^{(t)}} \mathbf{h}(\mathbf{x}^{(t)})\|_F^2 = \sum_j \sum_k \left(\frac{\partial h(\mathbf{x}^{(t)})_j}{\partial x_k^{(t)}} \right)^2 \left. \vphantom{\sum_j \sum_k} \right\} \text{encoder throws away all information}$$

↙
encoder keeps **only** good information

CONTRACTIVE AUTOENCODER

Topics: contractive autoencoder

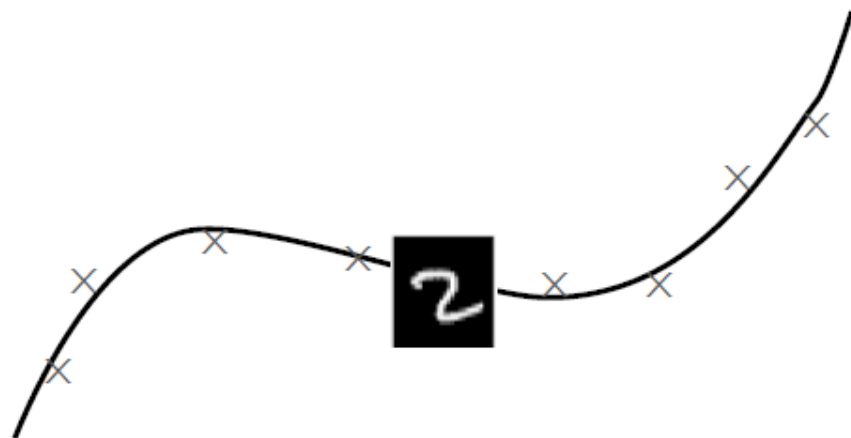
- Illustration:



CONTRACTIVE AUTOENCODER

Topics: contractive autoencoder

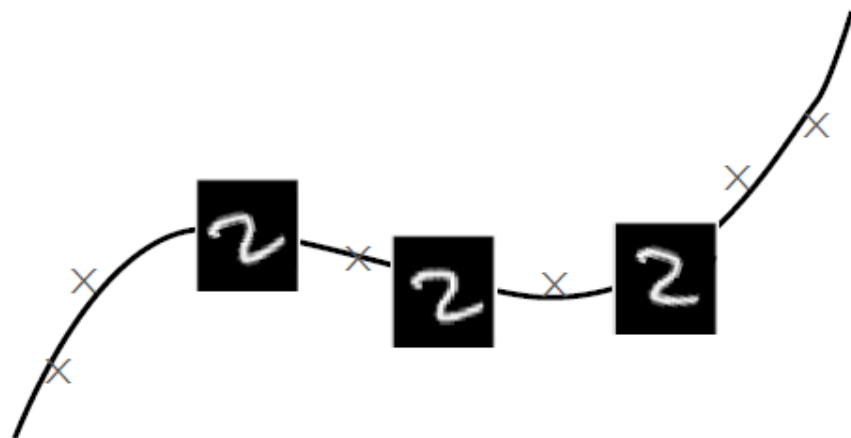
• Illustration:



CONTRACTIVE AUTOENCODER

Topics: contractive autoencoder

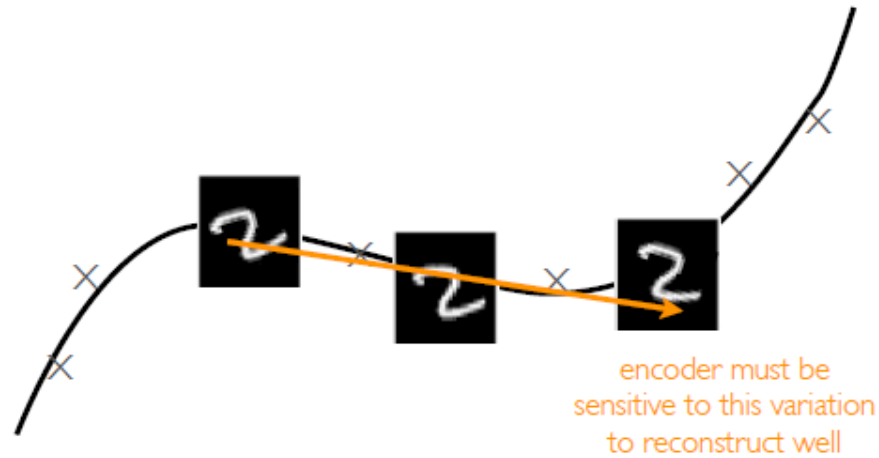
• Illustration:



CONTRACTIVE AUTOENCODER

Topics: contractive autoencoder

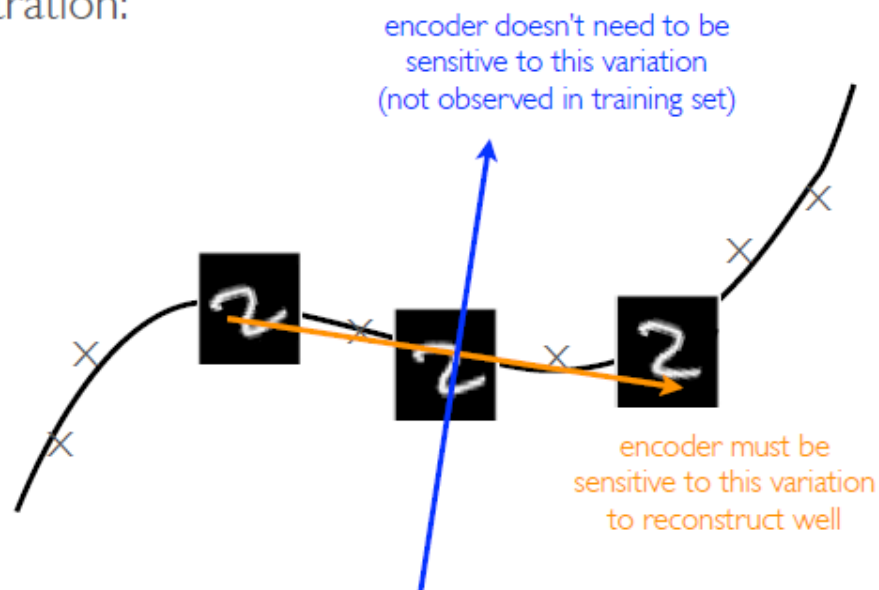
• Illustration:



CONTRACTIVE AUTOENCODER

Topics: contractive autoencoder

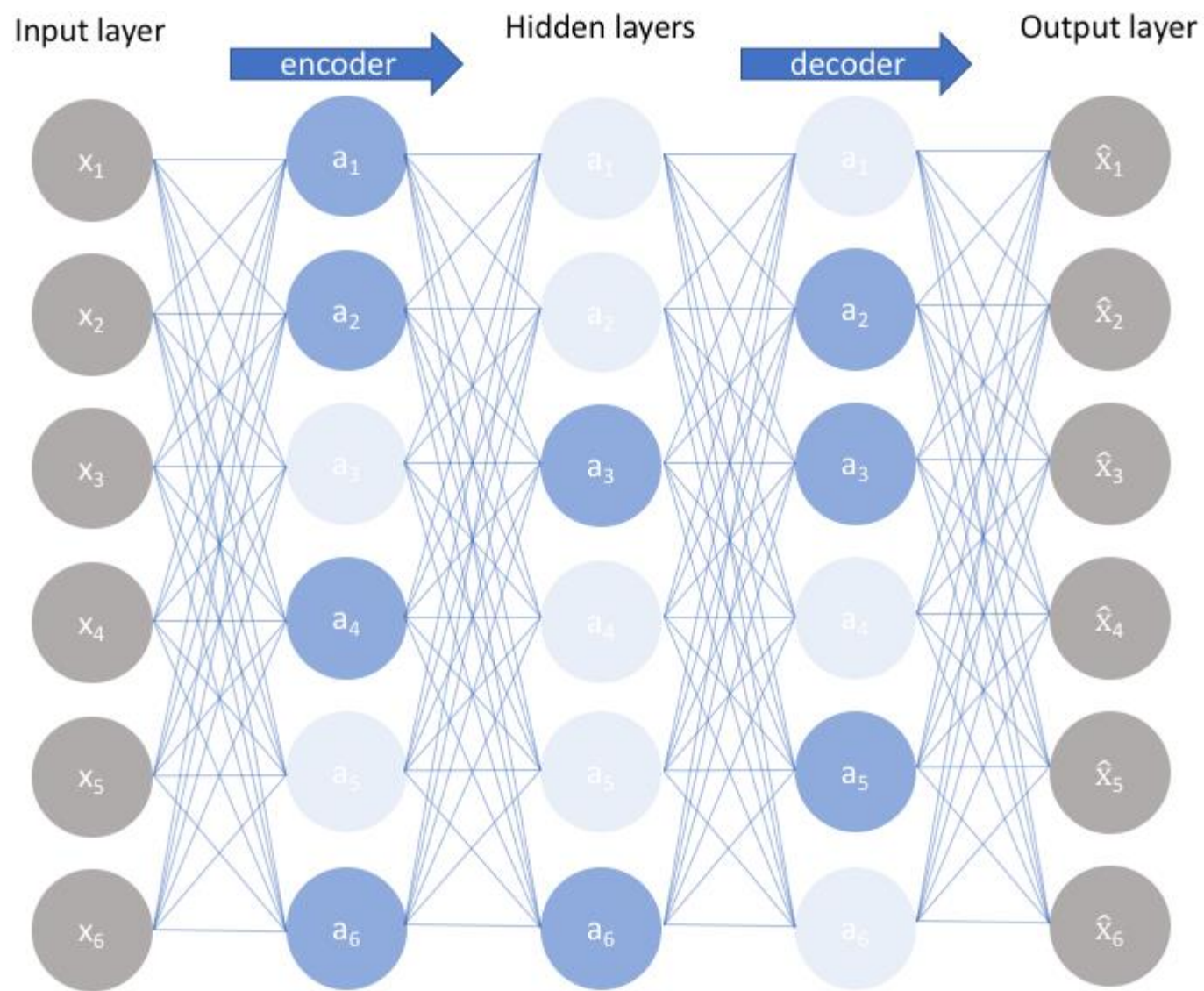
• Illustration:



Sparse autoencoders

Sparse autoencoders offer us an alternative method for introducing an information bottleneck without *requiring* a reduction in the number of nodes at our hidden layers. Rather, we'll construct our loss function such that we penalize *activations* within a layer. For any given observation, we'll encourage our network to learn an encoding and decoding which only relies on activating a small number of neurons. It's worth noting that this is a different approach towards regularization, as we normally regularize the *weights* of a network, not the activations.

A generic sparse autoencoder is visualized below where the opacity of a node corresponds with the level of activation. It's important to note that the individual nodes of a trained model which activate are *data-dependent*, different inputs will result in activations of different nodes through the network.



- **L1 Regularization:** We can add a term to our loss function that penalizes the absolute value of the vector of activations a in layer h for observation i , scaled by a tuning parameter λ .

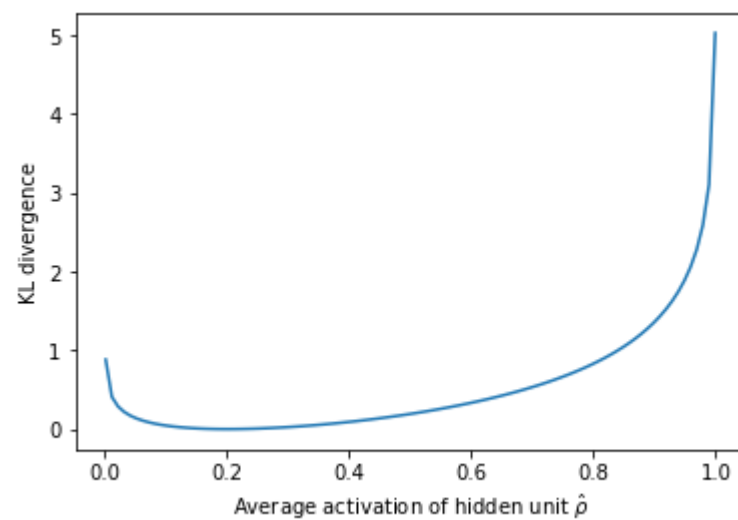
$$\mathcal{L}(x, \hat{x}) + \lambda \sum_i |a_i^{(h)}|$$

- **KL-Divergence:** In essence, KL-divergence is a measure of the difference between two probability distributions. We can define a sparsity parameter ρ which denotes the average activation of a neuron over a collection of samples. This expectation can be calculated as $\hat{\rho}_j = \frac{1}{m} \sum_i [a_i^{(h)}(x)]$ where the subscript j denotes the specific neuron in layer h , summing the activations for m training observations denoted individually as x . In essence, by constraining the average activation of a neuron over a collection of samples we're encouraging neurons to only fire for a subset of the observations. We can describe ρ as a Bernoulli random variable distribution such that we can leverage the KL divergence (expanded below) to compare the ideal distribution ρ to the observed distributions over all hidden layer nodes $\hat{\rho}$.

$$\mathcal{L}(x, \hat{x}) + \sum_j KL(\rho || \hat{\rho}_j)$$

$$\sum_{j=1}^{s_2} \rho \log \frac{\rho}{\hat{\rho}_j} + (1 - \rho) \log \frac{1 - \rho}{1 - \hat{\rho}_j}.$$

$$\text{KL}(\rho || \hat{\rho}_j) = 0 \text{ if } \hat{\rho}_j = \rho,$$



Summary

An autoencoder is a neural network architecture capable of discovering structure within data in order to develop a compressed representation of the input. Many different variants of the general autoencoder architecture exist with the goal of ensuring that the compressed representation represents *meaningful* attributes of the original data input; typically the biggest challenge when working with autoencoders is getting your model to actually learn a meaningful and generalizable latent space representation.

Summary

An autoencoder is a neural network architecture capable of discovering structure within data in order to develop a compressed representation of the input. Many different variants of the general autoencoder architecture exist with the goal of ensuring that the compressed representation represents *meaningful* attributes of the original data input; typically the biggest challenge when working with autoencoders is getting your model to actually learn a meaningful and generalizable latent space representation.

Because autoencoders *learn* how to compress the data based on attributes (ie. correlations between the input feature vector) *discovered from data during training*, these models are typically only capable of reconstructing data similar to the class of observations of which the model observed during training.

Summary

An autoencoder is a neural network architecture capable of discovering structure within data in order to develop a compressed representation of the input. Many different variants of the general autoencoder architecture exist with the goal of ensuring that the compressed representation represents *meaningful* attributes of the original data input; typically the biggest challenge when working with autoencoders is getting your model to actually learn a meaningful and generalizable latent space representation.

Because autoencoders *learn* how to compress the data based on attributes (ie. correlations between the input feature vector) *discovered from data during training*, these models are typically only capable of reconstructing data similar to the class of observations of which the model observed during training.

Applications of autoencoders include:

- Anomaly detection
- Data denoising (ex. images, audio)
- Image inpainting
- Information retrieval

WHICH AUTOENCODER ?

Topics: denoising autoencoder, contractive autoencoder

- Both the denoising and contractive autoencoder perform well

WHICH AUTOENCODER ?

Topics: denoising autoencoder, contractive autoencoder

- Both the denoising and contractive autoencoder perform well
 - Advantage of denoising autoencoder: simpler to implement
 - requires adding one or two lines of code to regular autoencoder
 - no need to compute Jacobian of hidden layer

WHICH AUTOENCODER ?

Topics: denoising autoencoder, contractive autoencoder

- Both the denoising and contractive autoencoder perform well
 - Advantage of denoising autoencoder: simpler to implement
 - requires adding one or two lines of code to regular autoencoder
 - no need to compute Jacobian of hidden layer
 - Advantage of contractive autoencoder: gradient is deterministic
 - can use second order optimizers (conjugate gradient, LBFGS, etc.)
 - might be more stable than denoising autoencoder, which uses a sampled gradient

Higher Order Contractive Auto-Encoder

Salah Rifai¹, Grégoire Mesnil^{1,2}, Pascal Vincent¹, Xavier Muller¹,
Yoshua Bengio¹, Yann Dauphin¹, and Xavier Glorot¹

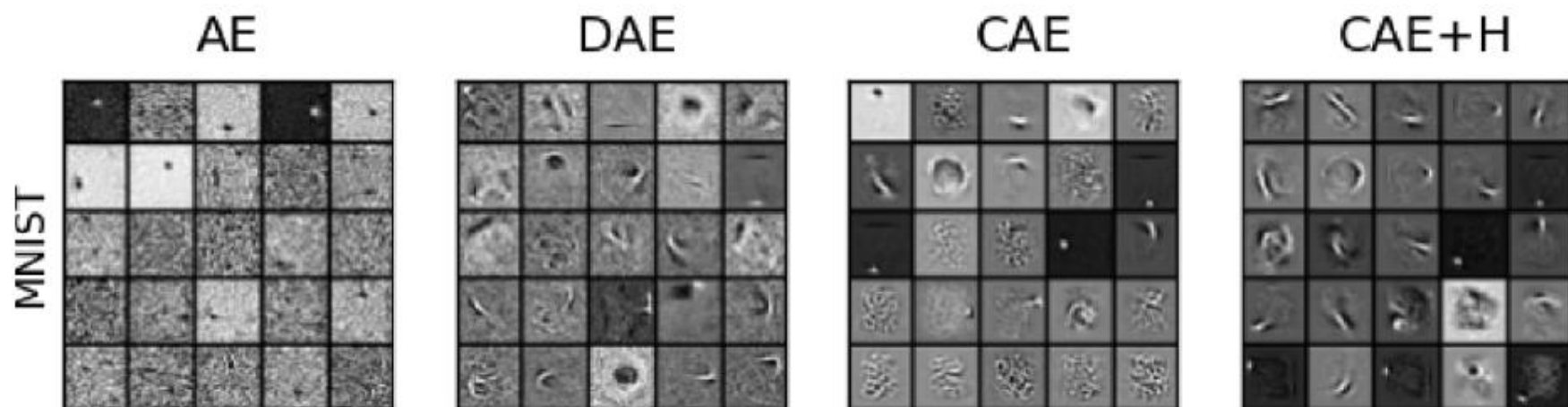
¹ Dept. IRO, Université de Montréal. Montréal (QC), H2C 3J7, Canada

² LITIS EA 4108, Université de Rouen. 768000 Saint Etienne du Rouvray, France

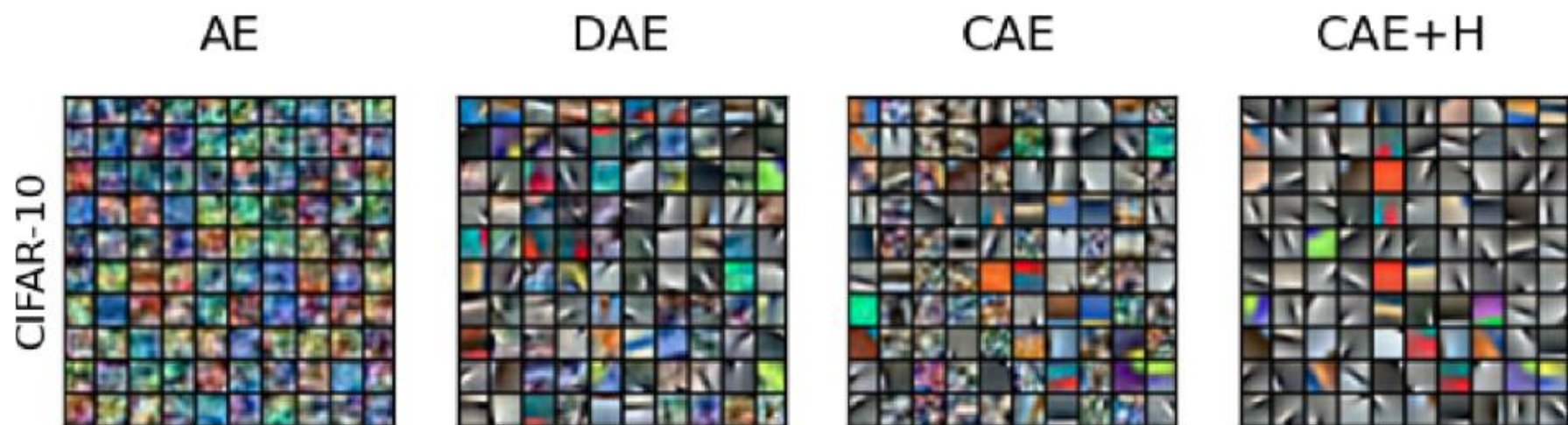
Abstract. We propose a novel regularizer when training an auto-encoder for unsupervised feature extraction. We explicitly encourage the latent representation to contract the input space by regularizing the norm of the Jacobian (analytically) and the Hessian (stochastically) of the encoder's output with respect to its input, at the training points. While the penalty on the Jacobian's norm ensures robustness to tiny corruption of samples in the input space, constraining the norm of the Hes-

<https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=2964d30862d0402b0d0ad4a427067f69e4a52130>

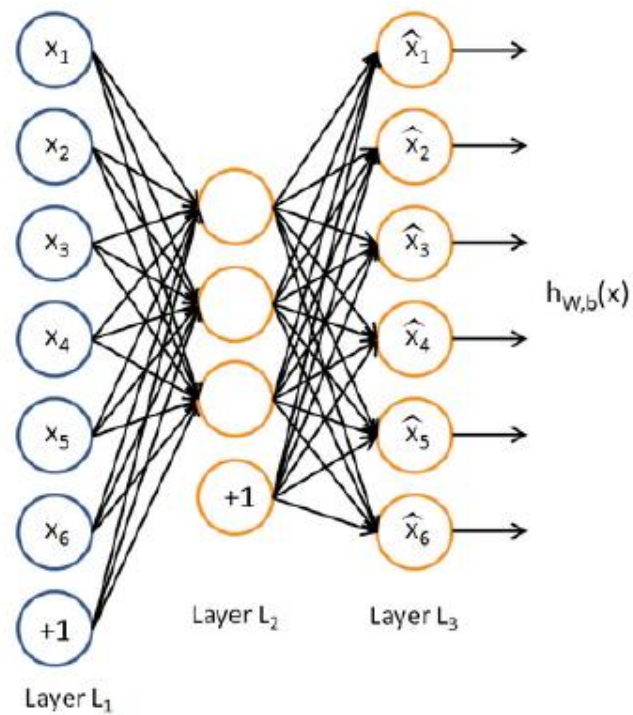
WHICH AUTOENCODER ?



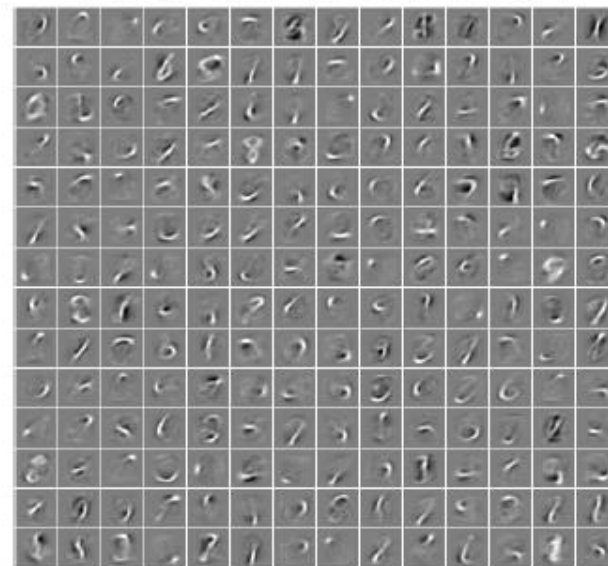
WHICH AUTOENCODER ?



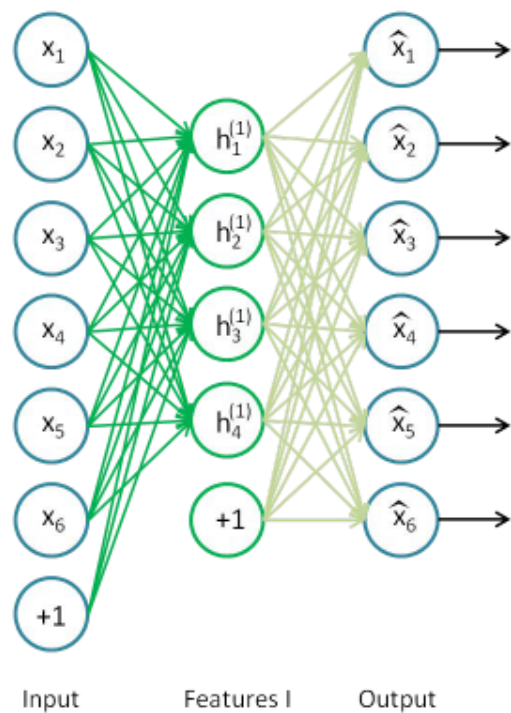
MORE HIDDEN LAYERS ?



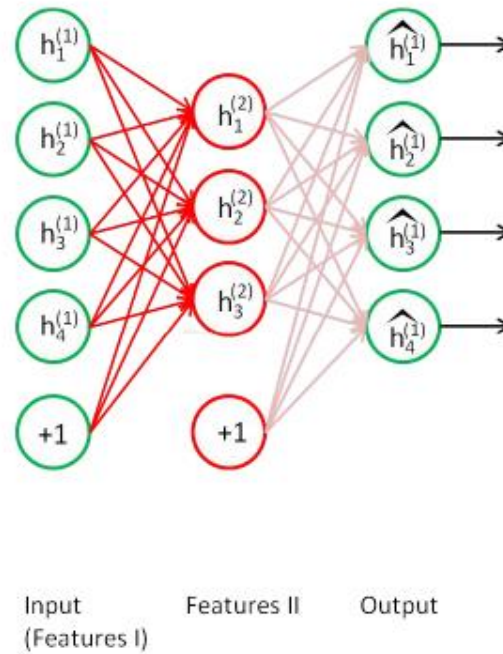
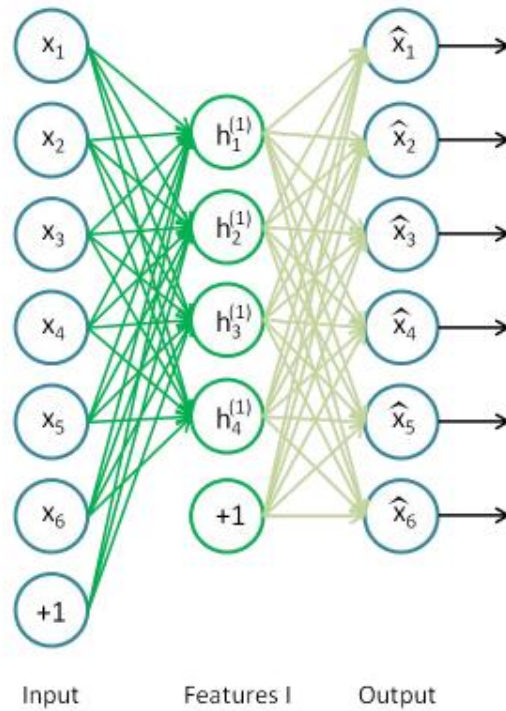
0	4	1	9	2	1	3	1	4	3
5	3	6	1	7	2	8	6	9	4
0	9	1	1	2	4	3	2	7	3
8	6	9	0	5	6	0	7	6	1
8	7	9	3	9	8	5	9	3	3
0	7	4	9	8	0	9	4	1	4
4	6	0	4	5	6	1	0	0	1
7	1	6	3	0	2	1	1	7	8
0	2	6	7	8	3	9	0	4	6
7	4	6	8	0	7	8	3	1	5



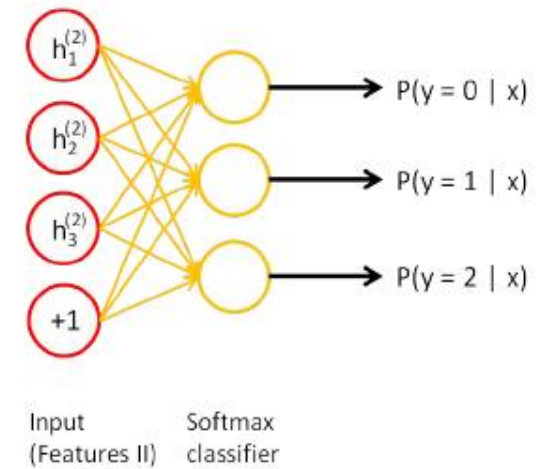
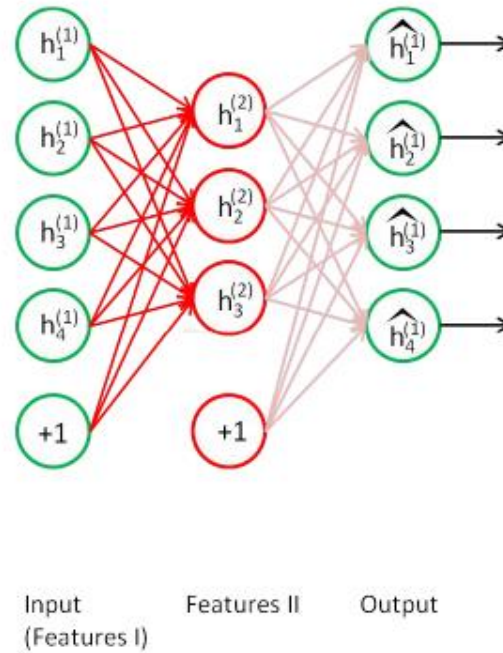
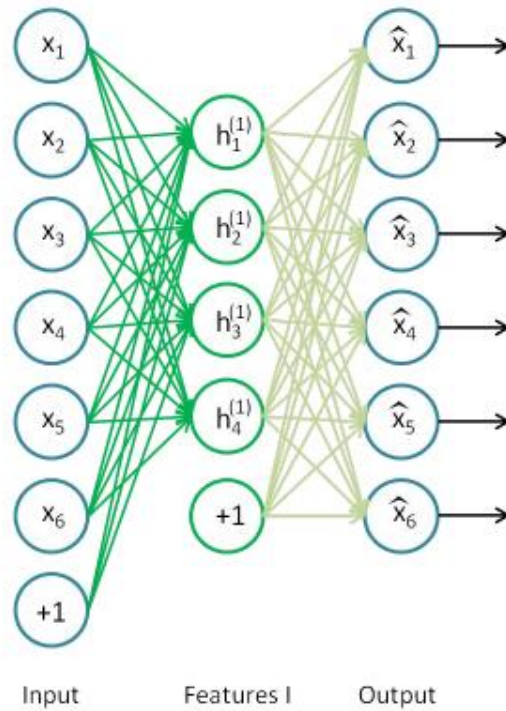
STACKED AUTOENCODERS



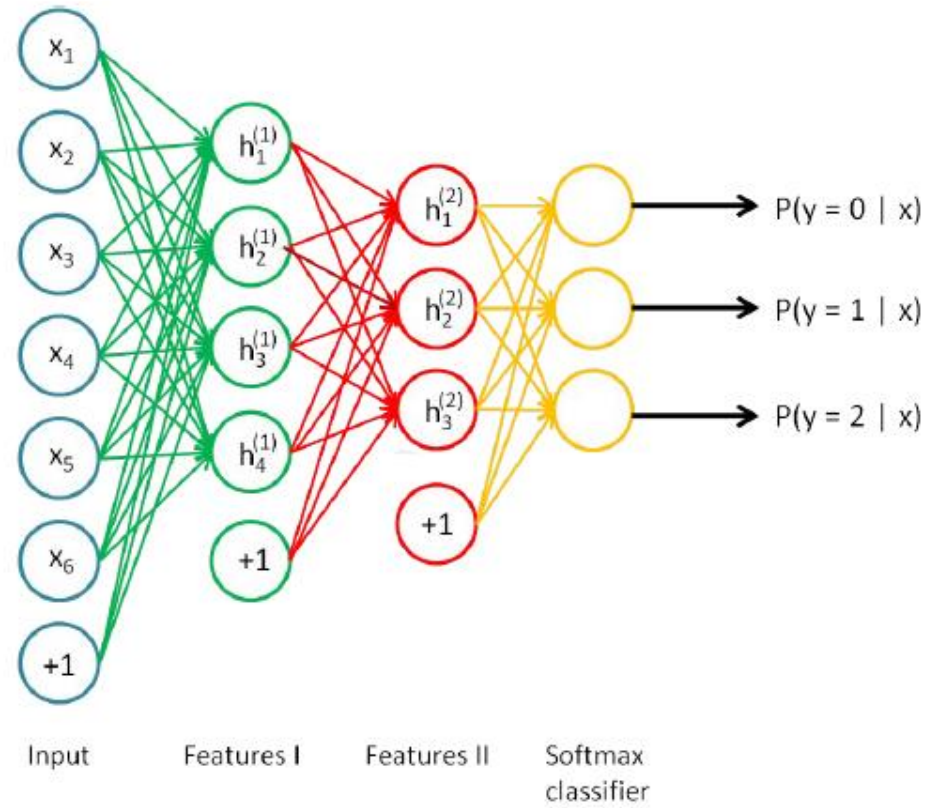
STACKED AUTOENCODERS



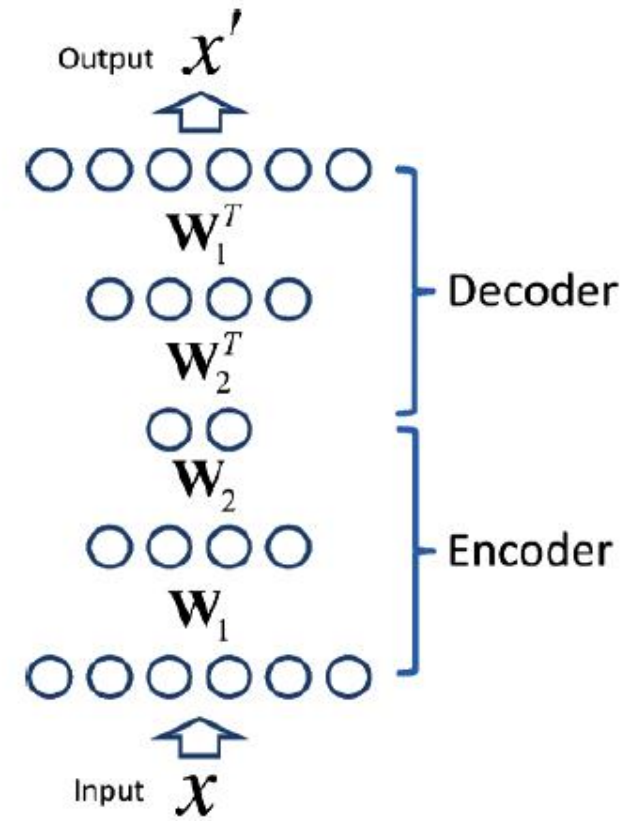
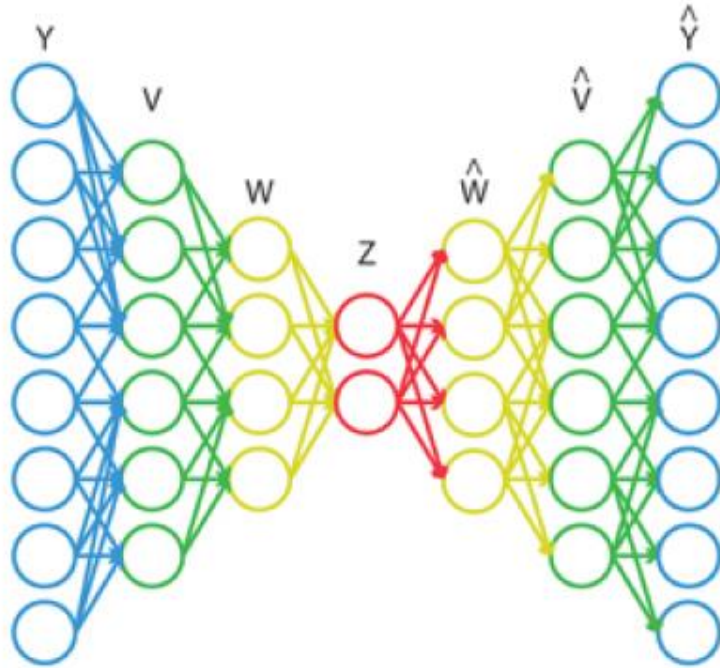
STACKED AUTOENCODERS



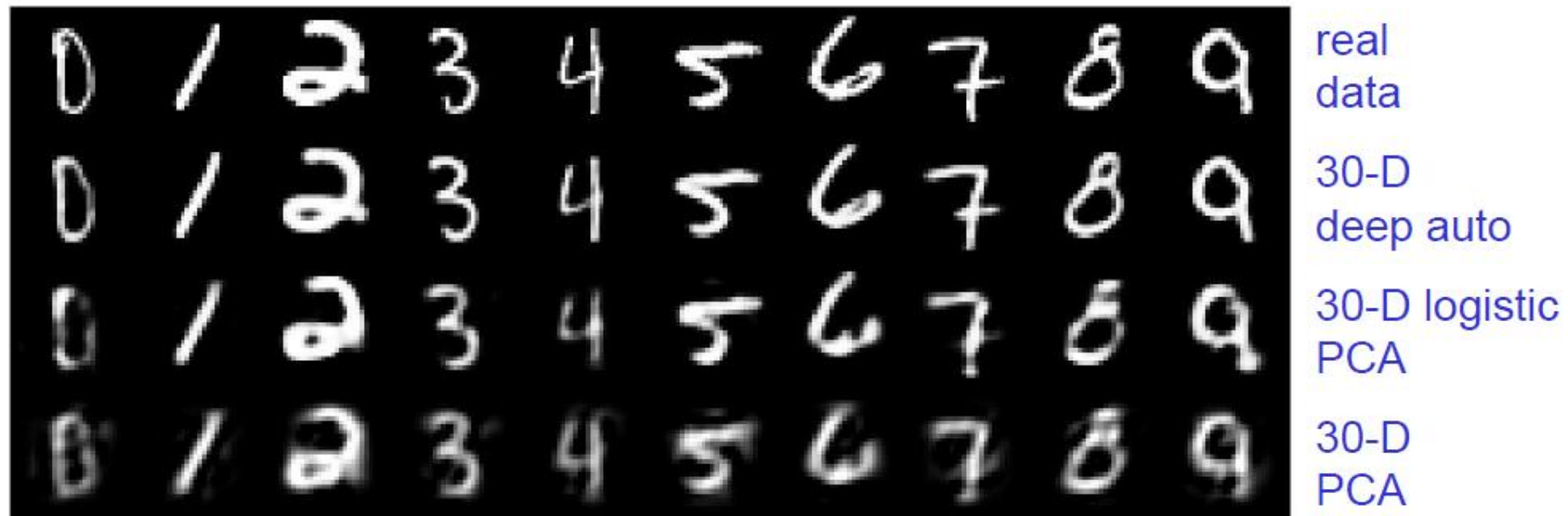
STACKED AUTOENCODERS



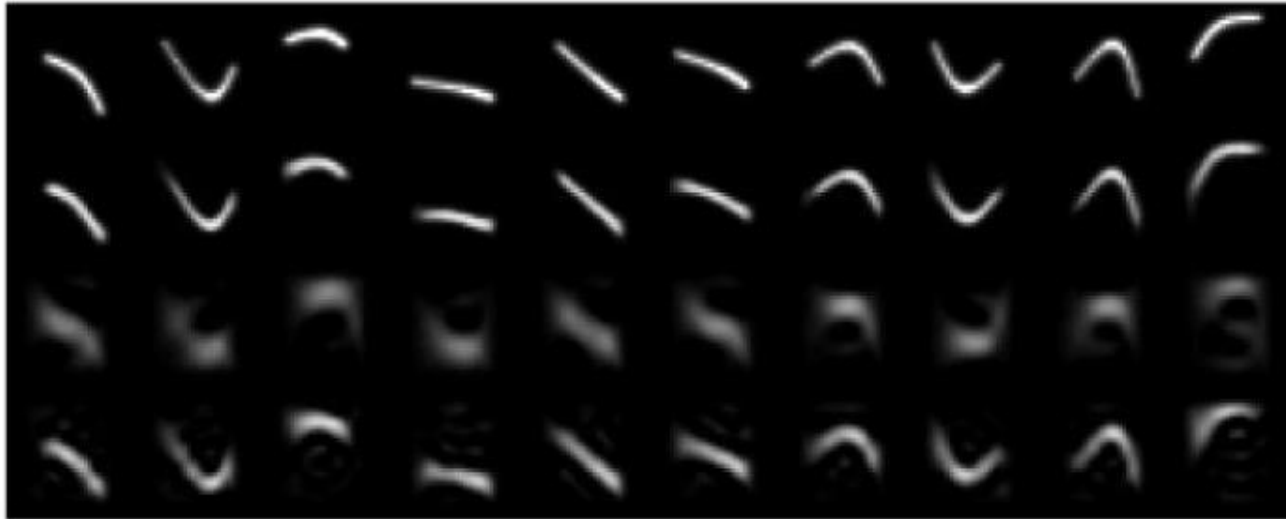
STACKED AUTOENCODERS



A comparison of methods for compressing digit images to 30 real numbers.



A very deep autoencoder for synthetic curves that only have 6 degrees of freedom



	squared error
Data	0.0
Auto:6	1.5
PCA:6	10.3
PCA:30	3.9

Reconstructions of face patches from new people

